Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

Vector Retrieval Drift under Corpus Growth: A Controlled Empirical Study with Metadata Filtering on EnterpriseRAG-Bench

BHARGAVA CHARY PEDDAPUDI[1], (Member, IEEE)
[1]Independent Researcher, San Francisco Bay Area, CA, USA (e-mail: pedapudibhargav@gmail.com)

Corresponding author: Bhargava Chary Peddapudi (e-mail: pedapudibhargav@gmail.com).

This work has no external funding to declare, and the author has no conflicts of interest to disclose. ORCID: https://orcid.org/0009-0002-8523-8415.

ABSTRACT Dense vector retrieval is a common first stage in enterprise retrieval-augmented generation (RAG), yet labeled Hit@k can fall as the index grows even when gold documents remain indexed. We measure this drift on EnterpriseRAG-Bench with gold-anchor pinning and exact cosine search. With `text-embedding-3-small`, raw Hit@10 falls $0.795 \to 0.585$ between 5k and 100k documents (drop 0.210, 95% CI $[0.155, 0.270]$; other schedules 0.200 to 0.250) and to 0.515 at the full $184{,}963$-document pool; Amazon Titan V2 falls $0.785 \to 0.610$ over the same range; the two drops are equivalent within ± 0.10, not ± 0.05. Label completeness is bounded only weakly: a pre-specified two-judge audit rated none of 420 replacement documents fully answering and 417 to 419 off-subject, while rating 80 to 98% of known gold documents on-subject but few fully answering. Chunked re-embedding changes the drop by -0.005 ($[-0.070, 0.060]$). BM25 on the text the dense model embedded loses 0.125 against 0.210; the interval of the difference excludes zero only for OpenAI. Reciprocal rank fusion adds $+0.11$ to $+0.15$ Hit@10 without removing the decline. An oracle source-type filter adds $+0.05$ to $+0.115$ up to 100k across primary-bank schedules and embedders (0.13 at the full pool); worst-case router errors of 9–13% erase the gain. The same `ef_search` setting ran an exact scan or HNSW depending on the database; per-size HNSW indexes at `ef_search`=200 lose 0.005 to 0.05 Hit@10 (OpenAI; ≤ 0.015 for Titan). Results describe these frozen stacks only.

INDEX TERMS Benchmark testing, Information retrieval, Metadata, Nearest neighbor methods, Retrieval augmented generation, Scalability, Semantic search

I. INTRODUCTION

ENTERPRISE retrieval-augmented generation (RAG) stacks typically embed documents into a vector index and retrieve top-k neighbors for generation [1], [2]. As organizations continuously ingest content, the index cardinality N increases even when historical answer documents remain indexed. Retrieval quality can drift downward without any label change. We call this phenomenon *vector retrieval drift under corpus growth*.

Prior information retrieval (IR) scaling work largely studies web-scale training or *model* capacity [3], [4]. A complementary line enlarges the *retriever corpus* to improve end-to-end RAG via answer-passage coverage [5]. That finding does *not* imply labeled first-stage Hit@k is monotone in N when gold documents are already indexed: coverage gains and near-neighbor crowding are distinct effects. Controlled measurements of enterprise document Hit@k under *gold-anchor pinning* as distractors grow remain scarce. Concurrent systems evaluations study filtered-search strategies inside production vector databases, including pgvector and PostgreSQL [6], [7], and query-wise recall inequality in vector similarity search [8], while industry deployments treat metadata filtering as an operational first-stage lever [9].

a: Research questions.

RQ1 How do labeled Hit@k and DocRecall@k decline as N grows under gold-anchor pinning, when every result is measured with exact (brute-force) cosine search?

RQ2 Does Okapi BM25 decline comparably on the same gold-pinned ladder when it indexes exactly the text the

dense model embedded, or is the dense decline larger?

RQ3 How large is the oracle metadata lift Δ_{meta} against an *exact* unfiltered baseline, does it change with N, and how much of it survives when the filter is chosen wrongly?

RQ4 Does the decline replicate under a second frozen embedder (Amazon Titan Text Embeddings V2) on the same pins, and within what margin are the two declines equivalent?

RQ5 How much of the measured quality loss do deployed approximate nearest-neighbor (ANN) indexes, such as hierarchical navigable small world (HNSW) graphs, add, and can the search mode (exact scan or HNSW) behind a reported number be taken for granted?

b: Contributions.

- A reproducible gold-pinned scale-ladder protocol on EnterpriseRAG-Bench [10] with exact search as the primary instrument, a documented document pool, several distractor schedules (round-robin, uniform-random, and composition-proportional up to the full pool), and banks of n=200 and n=400 questions.
- Exact-search drift curves for two embedders with paired bootstrap confidence intervals (CIs) (Hit@10 falls by 0.210 for OpenAI and 0.175 for Titan between 5k and 100k) and an equivalence analysis of the two declines.
- A metadata lift measured against an *exact* baseline (about $+0.08$ with no detectable trend up to 100k under round-robin on the primary bank, 0.060 on the n=400 bank at 100k, $+0.06$ to $+0.115$ across the OpenAI schedules) and a router-error simulation whose worst case erases the lift at an error rate of about 9–13%.
- A chunked-index control (overlapping chunks, max-chunk scoring) in which no change in the decline is detected, and a like-for-like BM25 control and a reciprocal-rank-fusion hybrid (Hit@10 $+0.11$ to $+0.15$) that does not remove the drift.
- A search-mode audit showing that a pgvector `ef_search` setting can run an exact scan on one database and an HNSW search on another, and a per-size HNSW study of recall, Hit@10 loss, latency, and capacity.
- A fail-closed validation harness, and a record of every correction relative to the earlier public release of this study (*v1*; Appendix F).

II. RELATED WORK

A. MODEL- VERSUS CORPUS-SIDE SCALING

Dense passage retrieval [2] and dual encoders [3] established neural first-stage retrieval. Fang *et al.* [4] fit power laws in retriever parameters and annotations. Heterogeneous zero-shot benchmarks [11] and embedding leaderboards [12] compare retrievers on fixed corpora; they do not vary N while holding labels fixed. Most directly related, Reimers and Gurevych [13] prove and confirm on MS MARCO that dense retrieval degrades faster than BM25 as an index seeded with only the relevant passages grows to 8.8M documents, using exact search and bi-encoders trained and dimension-varied for that corpus; their crossover, where BM25 overtakes dense, appears only between 10^6 and 8.8×10^6 passages for models trained without hard negatives. We study the same qualitative pressure under a protocol closer to enterprise deployment—an off-the-shelf commercial embedder, a production vector database (pgvector HNSW), and a company-document benchmark—and differ from [13] in four ways. (i) The corpus grows by nested distractor additions around a fixed set of pinned gold documents, with the pool and source mixture documented. (ii) Every headline number is an exact-search number, and a planner audit shows how approximate-labeled results can silently mix exact scans (Sec. V-F). (iii) Two commercial embedders are compared with an explicit equivalence margin instead of a failure to reject (Sec. V-I). (iv) We quantify what operational remedies recover: an oracle metadata filter against an exact baseline, the loss when the filter is chosen wrongly, and hybrid fusion of dense and BM25 rankings (Secs. V-C, V-L, V-K). In contrast to their crossover, BM25 leads the dense embedder at every size in point estimate, although the paired intervals include zero at the smaller sizes (Sec. V-H), so we claim no crossover. Weller *et al.* [14] show that the number of top-k document subsets that any single-vector embedding can return is limited by its dimension, and that current models fail on simple queries built to expose this, a possible reason for dense degradation that our experiments do not test. Our protocol freezes the embedder and keeps gold documents present, isolating crowding from coverage [5].

B. ANN SYSTEMS AND FILTERED SEARCH

HNSW [15] underpins production vector databases, alongside quantization- and disk-based alternatives such as FAISS [16] and DiskANN [17]; ANN-Benchmarks [18] standardizes recall–throughput comparisons at fixed dataset sizes. Vector database systems such as Milvus [19] expose attribute-filtered search, Filtered-DiskANN [20] builds label-aware graph indexes for filter predicates, and Pan *et al.* [21] survey vector database management systems. ACORN [22] extends HNSW with predicate-subgraph traversal so that search over an index stays effective under arbitrary filters, the problem that post-filtering an HNSW search by a size predicate exhibits in Sec. V-G. Recent systems work compares filtering strategies for approximate search in FAISS, Milvus, and pgvector [6], shows that filtered vector search inside a production PostgreSQL system behaves differently from specialized libraries [7], and documents query-wise recall inequality under mean-recall targets [8]. These motivate reporting Hit@$k(N)$ under fixed ANN hyperparameters and treating metadata filters as first-class experimental conditions, not only latency service-level objectives.

Filtered search is especially relevant to enterprise RAG, where queries carry tenant, product-line, or document-type constraints; oracle filters upper-bound the benefit of perfect filter prediction, and as each partition grows the same crowding pressure reappears inside it, which is what $\Delta_{\text{meta}}(N)$ quantifies.

C. ENTERPRISE RAG EVALUATION

EnterpriseRAG-Bench [10] provides company-internal-style documents and questions with expected document IDs. Industry deployments report knowledge-base Hit@k with metadata scoping [9]. We use EnterpriseRAG-Bench (ERB) labels for a retrieval-only scale ladder, complementary to end-to-end answer scoring such as reference-free RAG evaluation [23] and the designs catalogued in RAG surveys [24]; sparse-versus-dense scaling comparisons [25] motivate the lexical baseline.

Our evaluation stops at labeled document retrieval: generation faithfulness, citation accuracy, and multi-hop reasoning inherit first-stage failures but add judge noise.

D. POSITIONING

The qualitative effect that dense retrieval can degrade faster than BM25 as an index grows under exact search is established by Reimers and Gurevych [13]; our contribution is incremental on that point and lies in the enterprise setting, the controls and audits (notably the search-mode audit), and the study of what metadata filtering and hybrid fusion recover.

III. PROBLEM FORMULATION

Let $\mathcal{D}_N$ be a corpus of N documents with fixed embeddings $e(d) \in \mathbb{R}^m$. For query q with gold set $G(q) \subseteq \mathcal{D}_N$, a retriever returns $R_k(q) \subset \mathcal{D}_N$ of size k. We measure

$$\mathrm{Hit@}k = \mathbf{1}[G(q) \cap R_k(q) \neq \emptyset], \tag{1}$$

$$\mathrm{DocRecall@}k = |G(q) \cap R_k(q)|/|G(q)|, \tag{2}$$

$$\mathrm{MRR@10} = 1/\min\{i \leq 10 : r_i \in G(q)\}, \tag{3}$$

averaged over the question bank $\mathcal{Q}$, with MRR@10 (mean reciprocal rank at 10) $= 0$ when no gold document is in the top ten (the stored lists have ten entries, so MRR below always means MRR@10, and we write MRR for short). Differences in tables are computed from unrounded values, so a displayed triple can differ from its rounded parts by 0.001.

a: Scale construction.

Gold anchors (the union of all $G(q)$) receive ranks $0..A-1$. The remaining embedded documents (Sec. IV-A) are ordered by a seeded round-robin over `source_type` strata (seed 42) and receive the following ranks. The corpus at scale N is $\{d : \mathrm{scale_rank}(d) < N\}$, so the ladders are nested. This matches the operational constraint that answer documents remain indexed as the enterprise corpus grows. Because the round-robin exhausts small strata early, the source-type composition of the distractors changes along the ladder; Sec. V-J repeats the study with uniform-random orderings of the same documents.

b: Conditions.

(1) **raw**: cosine search over all d with $\mathrm{scale_rank} < N$. (2) **meta_in** (the oracle filter, written "meta" in the text): the same search restricted to documents whose `source_type` belongs to the set of types attached to the question. This is an *idealized* router that never errs; Sec. V-L relaxes it. On the primary-200 bank this set coincides with the source types of the gold documents; on the n=400 bank it does so for all but one question, one of whose three gold documents is a Confluence page outside its listed types (this changes document recall by $1/1200$ and leaves Hit@10 and MRR unchanged). (3) **meta_first**: the filter used in the v1 release, which kept only the first listed type; it is reported for continuity (Appendix A). Unless stated otherwise, *all conditions are evaluated with exact search* (Sec. IV).

c: Trend fits and lift constancy.

We fit $\mathrm{Hit@}k(N) \approx a + b \ln N$ as a descriptive trend on this ladder (negative b is decline). Metadata utility is

$$\Delta_{\mathrm{meta}}(N) = \mathrm{Hit@10}_{\mathrm{meta}}(N) - \mathrm{Hit@10}_{\mathrm{raw}}(N), \tag{4}$$

with both terms measured by exact search so that the lift is not confounded by approximate-search loss. Whether the lift changes with N is tested directly: we report $\Delta_{\mathrm{meta}}(100\mathrm{k}) - \Delta_{\mathrm{meta}}(5\mathrm{k})$ and the slope of Δ_{meta} against $\log_{10} N$, each with a paired bootstrap interval. The operational threshold used in the v1 release, $N^\star(\tau) = \min\{N : \Delta_{\mathrm{meta}}(N') < \tau \; \forall N' \geq N\}$ with τ=0.10, is retained only as a sensitivity quantity (Appendix A): it is not stable across question banks of different size, so we do not use it as a result.

IV. EXPERIMENTAL SETUP

A. DATASET, DOCUMENT POOL, AND QUESTIONS

ERB v1.0.0 documents (`dsid_*`) and questions are filtered to non-empty `expected_doc_ids`, excluding `high_level` and `info_not_found` [10]. The primary evaluation set is a stratified sample of **200** questions by `question_type` (seed 42), spanning nine types (largest: basic n=63, semantic n=45, metadata n=39). A larger bank of n=400 extends it with 200 further stratified questions (Sec. V-J). Query embeddings are computed once and cached under a key that includes the question type, because the same benchmark question identifier can occur with several question types.

a: Document pool.

The benchmark contains 511,962 documents; we downloaded and extracted 184,966 manifest entries (184,963 distinct documents: 627 gold anchors and 184,336 distinct distractors, three distractor identifiers occurring twice) and embedded the first 99,998 documents of the schedule below (627 gold anchors and 99,371 distractors); the main ladders use only these documents, and Sec. V-J adds ladders over the full extracted pool, whose remaining documents were embedded afterwards. The extracted pool is the part of the benchmark that we downloaded, not a designed sample: Gmail and Slack are each cut at 40,000 documents (eight archive slices each), the other seven sources are complete except for four documents missing from the extraction relative to the benchmark's published counts (Jira 6,118 of 6,120, Google Drive 25,107

TABLE 1. Composition (% of the corpus, gold anchors included) of the 99,998 embedded documents at each ladder size under the round-robin schedule; document counts per source are in Table 17; the "Pool" column is the number of documents extracted per source (184,966 in total).

Source	Pool	5k	25k	50k	75k	100k
confluence	5,192	13.7	11.6	10.4	6.9	5.2
fireflies	10,173	10.4	11.0	11.2	12.4	10.2
github	8,052	11.0	11.1	11.2	10.7	8.1
gmail	40,000	10.1	10.9	11.1	12.3	14.1
google_drive	25,107	11.2	11.1	11.2	12.4	14.1
hubspot	15,016	10.7	11.0	11.2	12.4	14.1
jira	6,118	11.9	11.3	11.3	8.2	6.1
linear	35,308	11.1	11.1	11.2	12.4	14.1
slack	40,000	10.0	10.9	11.1	12.3	14.1
Total	184,966	100	100	100	100	100

of 25,108, HubSpot 15,016 of 15,017) and three duplicated Confluence entries (5,192 entries, 5,189 distinct identifiers). A further 108 benchmark questions were dropped because at least one gold document is absent from the pool, leaving 462 usable questions from which both banks are drawn. Three Confluence identifiers occur twice in the manifest at different ranks and the database keeps one row each (the original copy kept a later rank for three of them, so it has 99,997 distinct documents below rank 100,000 and the fresh restore 99,998); exact search on a byte copy of the original database and on the fresh restore returns identical lists for all 4,800 question-by-condition lists at 5k, 50k, and 100k (both embedders), so this does not affect any result. Results describe growth within the extracted pool and should not be read as a statement about the full benchmark.

b: Distractor schedule.

Gold anchors take ranks 0..626. Distractors are shuffled within each `source_type` (seed 42) and then drawn in round-robin order over the nine sources. Small sources are therefore exhausted early (Confluence at rank ≈46k, Jira ≈54k, GitHub ≈68k, Fireflies ≈80k) and the mixture changes along the ladder: at 5k each source contributes 10–14% of the corpus, at 100k Confluence, Jira, GitHub, and Fireflies contribute 5, 6, 8, and 10% and the other five 14% each (Table 1). These four exhausted sources hold 113 of the 200 primary questions under their first listed type, so the schedule is not neutral for the metadata condition. We therefore repeat the study under uniform-random orderings of the same embedded documents (Sec. V-J).

B. SEARCH EXECUTION AND ENVIRONMENT

a: Primary measurement: exact search.

All headline numbers use exact cosine search: `ORDER BY embedding <=> q, doc_id LIMIT 10` over a single table, with the predicate scale_rank $< N$ (and `source_type` in the oracle set for the meta condition), executed with index and bitmap scans disabled for the session so that PostgreSQL scans and sorts. Exact search is deterministic and independent of index construction, which is why we make it the primary instrument. For each embedder and condition an `EXPLAIN` on probe queries at three scales confirms that no HNSW index scan occurred, and the harness repeats this check whenever a result file claims a search mode.

b: Approximate search.

pgvector [26] HNSW uses m=16, ef_construction=64, cosine distance (`vector_cosine_ops`). We study two approximate configurations. (i) *Single-index*, the v1 configuration: one HNSW graph over the whole table, 103,033 OpenAI rows (the ladder documents plus 3,035 pool documents ranked beyond the ladder; 3,036 on the original copy) and 99,998 Titan rows, queried with ef_search=200 and `hnsw.iterative_scan=relaxed_order`, with scale_rank $< N$ as a predicate; the same table also carries B-tree indexes on `scale_rank` and `source_type` (`idx_chunks_scale_rank`, `idx_chunks_source_type` and their Titan counterparts), which the planner can use instead of the HNSW index. (ii) *Per-size*: for every size N a separate table of exactly the first N documents with its own HNSW index, as in deployment (Sec. V-G). In either configuration PostgreSQL decides for each statement whether to use the HNSW index or an exact scan; Sec. V-F shows that this decision depends on N, ef_search, and table statistics, which is why the search mode behind a result must be verified rather than assumed.

c: Environment.

PostgreSQL 16.15 and pgvector 0.8.6 in a Docker container (aarch64 Linux, 14 central processing unit (CPU) cores and 7.75 GiB assigned) on an Apple M4 Pro host (48 GiB). The exact ladders and per-size indexes were run on a fresh restore of the database dumps with `shared_buffers`=2 GB; a frozen byte copy of the original database (default 128 MB buffers; it had received 8.3×10^5 in-place row updates and no manual `VACUUM` or `ANALYZE`) was used to reproduce and explain the v1 HNSW numbers.

C. EMBEDDERS, TEXT, AND TRUNCATION

The primary embedder is OpenAI `text-embedding-3-small` (d=1536) [27]; the secondary is Amazon Titan Text Embeddings V2 (`amazon.titan-embed-text-v2:0`, d=1024, unit-normalized) [28], stored in a separate table with the same design. Documents are embedded once as whole documents (title plus body), so ERB gold labels, which are document identifiers, become a direct membership test without a passage-to-document step (Sec. V-J reports a chunked control). Inputs beyond a model's limit were truncated from the end: the OpenAI arm at 8,191 tokens (cl100k_base, verified with the official tokenizer; 11 documents affected) and the Titan arm at a conservative 24,000-character client-side cap (90 documents, 5 of them gold anchors). Table 2 gives the length distribution of the 99,998 ladder documents and of the gold documents; Sec. V-I shows that the Titan

Algorithm 1 Gold-pinned exact-search scale ladder

Require: Documents $\mathcal{D}$, question bank $\mathcal{Q}$, scales $\mathcal{N}$, conditions $\{\mathrm{raw}, \mathrm{meta}\}$

Assign gold anchors ranks $0..A-1$; order the distractors (round-robin over `source_type` or uniform random; seed 42 or 0)
Embed each $d \in \mathcal{D}$ once and each question once (vectors cached and fingerprinted)
for $N \in \mathcal{N}$ **do**
    Restrict the searchable set to $\{d : \mathrm{scale_rank}(d) < N\}$
    for condition $c \in \{\mathrm{raw}, \mathrm{meta}\}$ **do**
        for question $q \in \mathcal{Q}$ **do**
            Exact top-k by cosine under c (ties by identifier); store the list and scores; record Hit@1/5/10, DocRecall@10, MRR
        end for
    end for
end for
Check list invariants (containment, nested corpora, score monotonicity) and the execution plans
Bootstrap intervals; $\Delta_{\mathrm{meta}}(N)$ with paired intervals; fit $a + b \ln N$; test lift change and equivalence

truncation touches only two primary questions and changes no conclusion.

D. LADDER, METRICS, AND CONFIDENCE INTERVALS

$N \in \{5\mathrm{k}, 10\mathrm{k}, 15\mathrm{k}, 20\mathrm{k}, 25\mathrm{k}, 40\mathrm{k}, 50\mathrm{k}, 75\mathrm{k}, 100\mathrm{k}\}$, evaluation k=10. The per-question retrieved lists and scores of every run are stored, so that metrics, invariants, and statistics can be recomputed from the lists alone. Intervals are percentile bootstrap [29] intervals over question indices (B=10,000, seed 42; nominal intervals without multiplicity adjustment, so "significant" below means only that an interval excludes zero) computed from the per-question flags of the run being described; differences between conditions, scales, or embedders resample the same questions for both terms (paired), two-sided 95% intervals are reported for drops, lifts, and levels, and equivalence statements use 90% intervals. Fits use the planned ladder points only.

E. VALIDATION LAYERS

L1: labeled IR (primary claims); L2: integrity checks (unique eval IDs, Hit@k membership, empty-retrieval gate, embedding coverage, and, in this revision, the list invariants and the harness of Appendix B). Two audits of the v1 runs remain secondary and carry no weight in any result of this paper: (L3) an exploratory `gpt-4o-mini` relevance score on a 40-question subsample, and (L4) a large language model (LLM)-assisted (`gpt-4o`) audit of 62 high-priority rows, which judged the gold text adequate to answer the question on 49 rows and inadequate on 13 (a manual spot-check of 10 rows agreed with the LLM label on 9; artifacts `L4_LLM_TRANSPARENCY` and `L4_SPOTCHECK_HUMAN`); this shows that some gold labels are loose and is not human labeling of all rows. Primary claims in the abstract and conclusion rest on L1; the label audit is reported as a model-judged sensitivity check.

V. RESULTS

A. EXACT-SEARCH LADDER (PRIMARY RESULT)

Table 3 and Fig. 1 give the primary result: exact cosine search over the gold-pinned ladder (unique eval IDs; Hit@k recomputed as gold∈top-k). Raw Hit@1 falls from 0.600 to 0.340, MRR from 0.658 to 0.408, and Hit@10 from 0.795 at 5k to 0.585 at 100k; the Hit@10 drop is 0.210 with a paired bootstrap 95% interval of $[0.155, 0.270]$ (Table 4). In relative terms the decline is largest for the rank-sensitive metrics: Hit@1 loses 43% and MRR 38% of their 5k value, versus 26% for Hit@10. The number of questions whose gold document is outside the top ten rises from 41 to 83 of 200, while the mean gold rank among the successes moves only from 1.86 to 2.61, so the dominant failure is a gold document leaving the top ten rather than sliding within it. Capacity reports should therefore publish Hit@1 and MRR alongside Hit@10. The v1 release reported a steeper early step between 5k and 10k (0.795→0.660); under exact search the step is 0.795→0.735, and Sec. V-F shows that the excess came from the query planner switching from an exact scan to the approximate index between those two sizes.

Under exact search with nested corpora and pinned gold documents, adding distractors can only keep or displace a question's gold documents, so a decline is expected; what the study measures is its size, its rate, and what mitigates it. DocRecall@10 tracks Hit@10 closely because most questions have one or two gold documents. Bootstrap intervals in Table 3 use the per-question flags of this sweep only. The n=400 bank gives the same picture (0.7725→0.5675, drop 0.205, $[0.1675, 0.245]$; Table 4). Amazon Titan V2 on the same pins declines by 0.175 (0.785→0.610, $[0.125, 0.230]$); its full ladder is in the released artifacts (Appendix C), and Sec. V-I compares the two embedders formally.

a: Rank depth.

The stored exact top-100 lists show where gold documents go. Hit@20, Hit@50, and Hit@100 fall from 0.835, 0.890, and 0.940 at 5k to 0.640, 0.690, and 0.755 at 100k. Of the 42 questions whose gold leaves the top ten, the best gold document is within rank 20 for 11, within 50 for 21, and within 100 for 34 (median 29.5 among the 34 within rank 100); 8 fall beyond 100. Even a perfect reranker over the top 100 could therefore recover at most 34 of the 42 displaced questions.

B. SCALING FITS

We fit $\mathrm{Hit@}k(N) \approx a + b \ln N$ to the nine ladder points as a descriptive trend. For raw OpenAI Hit@10 the slope is b=−0.068 per unit $\ln N$ (95% bootstrap interval $[-0.088, -0.050]$, R^2=0.988), essentially unchanged when the 5k point is dropped (−0.065), so the trend is not an artifact of the first step; the slopes of Hit@1 and MRR are −0.082 and

TABLE 2. Length in characters of the embedded text (the title is part of it) of the 99,998 ladder documents and of the 272 gold documents of the primary bank, the share above the BM25 caps (4,000 and 8,191 characters; 72.0% and 21.1% of the ladder documents for the title-plus-text BM25 input, which repeats the title), and the counts at or above the OpenAI cap (8,191 cl100k tokens; 11 documents, one a gold anchor, none a primary-bank gold document) and the Titan cap (24,000 characters).

Population	n	Median	p90	Max	>4k chars (%)	>8,191 chars (%)	≥8,191 tok.	>24k chars
Ladder documents	99,998	5,543	10,058	34,284	71.4	20.7	11	90
Primary-200 gold	272	5,354	11,187	28,602	67.6	24.3	0	2

TABLE 3. OpenAI exact vector search (primary-200): raw and `meta_in` across corpus scales.

Condition	N	Hit@1	Hit@5	Hit@10	Hit@10 95% CI	DocR@10	MRR
`raw`	5k	0.600	0.740	0.795	[0.735, 0.850]	0.765	0.658
`raw`	10k	0.485	0.670	0.735	[0.670, 0.795]	0.696	0.576
`raw`	15k	0.470	0.655	0.710	[0.645, 0.770]	0.672	0.554
`raw`	20k	0.465	0.645	0.680	[0.615, 0.745]	0.642	0.539
`raw`	25k	0.450	0.630	0.670	[0.605, 0.735]	0.632	0.523
`raw`	40k	0.400	0.580	0.645	[0.575, 0.710]	0.603	0.473
`raw`	50k	0.370	0.570	0.640	[0.570, 0.705]	0.597	0.451
`raw`	75k	0.355	0.525	0.595	[0.525, 0.660]	0.552	0.425
`raw`	100k	0.340	0.505	0.585	[0.515, 0.650]	0.540	0.408
`meta_in`	5k	0.680	0.825	0.875	[0.825, 0.920]	0.857	0.741
`meta_in`	10k	0.580	0.760	0.835	[0.780, 0.885]	0.812	0.664
`meta_in`	15k	0.565	0.740	0.805	[0.750, 0.860]	0.774	0.643
`meta_in`	20k	0.535	0.730	0.770	[0.710, 0.825]	0.741	0.615
`meta_in`	25k	0.525	0.715	0.760	[0.700, 0.820]	0.730	0.602
`meta_in`	40k	0.455	0.680	0.720	[0.655, 0.780]	0.687	0.544
`meta_in`	50k	0.435	0.645	0.715	[0.650, 0.775]	0.678	0.526
`meta_in`	75k	0.410	0.615	0.685	[0.620, 0.750]	0.646	0.498
`meta_in`	100k	0.390	0.610	0.665	[0.595, 0.730]	0.626	0.478

TABLE 4. Corpus-scale drift (5k→100k) for Hit@10 under raw and `meta_in`.

Arm	Condition	Hit@10 (5k)	Hit@10 (100k)	ΔHit@10	95% CI
OpenAI n=200	`raw`	0.795	0.585	0.210	[0.155, 0.270]
OpenAI n=200	`meta_in`	0.875	0.665	0.210	[0.155, 0.270]
Titan n=200	`raw`	0.785	0.610	0.175	[0.125, 0.230]
Titan n=200	`meta_in`	0.870	0.685	0.185	[0.135, 0.240]
OpenAI n=400	`raw`	0.7725	0.5675	0.205	[0.1675, 0.245]
OpenAI n=400	`meta_in`	0.850	0.6275	0.2225	[0.1825, 0.2625]

-0.082. For Titan the slopes are -0.087 (Hit@1), -0.064 ($[-0.084, -0.044]$, Hit@10), and -0.074 (MRR), and for the n=400 bank -0.069, -0.067, and -0.073; all $R^2 \geq 0.955$. The fits are summaries of these nine points, not claimed scaling laws, and extrapolation beyond 100k is not supported: the full-pool ladders of Sec. V-J fall faster after 100k (0.070 from 100k to 184,963 documents) than the fit implies (0.042).

C. METADATA LIFT AGAINST AN EXACT BASELINE

We define $\Delta_{\text{meta}}(N)$ as the exact oracle-filtered Hit@10 minus the exact unfiltered Hit@10, so that approximate-search loss cannot inflate it. For OpenAI it is $+0.080$ at 5k and at 100k and lies between $+0.075$ and $+0.100$ at every size; every paired 95% interval excludes zero (Table 5 and Fig. 2). Titan gives the same picture ($+0.070$ to $+0.115$). The v1 release reported $+0.130$ at 100k for OpenAI; that figure compared a metadata run with an approximate baseline that had lost 0.075 Hit@10 (Sec. V-F), so the exact-search lift is smaller than first reported.

The lift does not change measurably with N. Between 5k and 100k it changes by 0.000 (90% interval $[-0.040, +0.040]$) for OpenAI at n=200 and by -0.0175 ($[-0.045, +0.010]$) at n=400, and both intervals lie inside the margin ± 0.05 (Appendix D); the slope against $\log_{10} N$ at n=400 is -0.017 (90% interval $[-0.041, +0.006]$). For Titan the change is -0.010 ($[-0.055, +0.035]$), equivalent within ± 0.10 but not provably within ± 0.05. We therefore detect no trend in the lift up to 100k under round-robin (0.06–0.115 across all arms and sizes), but a change of up to 0.05 cannot be excluded, and the lift is larger at the full pool (Sec. V-J). The legacy first-listed-type filter yields lifts within 0.005 of the all-types filter at every size of the round-robin ladders and within 0.010 on the uniform ladders (Appendix A), so that implementation detail changes no lift by more than 0.010.

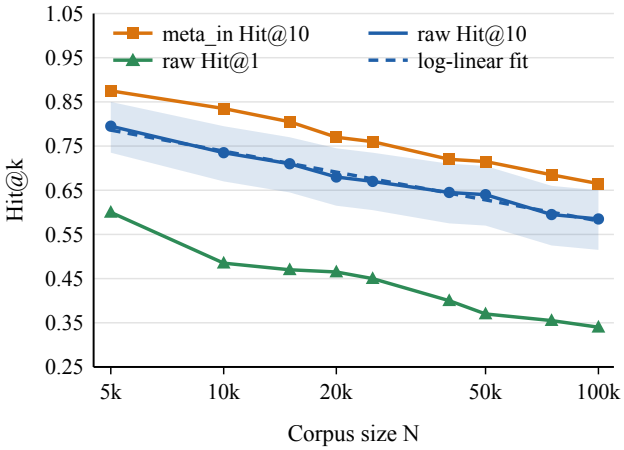


FIGURE 1. **OpenAI arm, exact search: raw and oracle-filtered (`meta_in`) Hit@10 and raw Hit@1 versus corpus size *N* (log scale), with the log-linear fit of raw Hit@10 (dashed). Shaded band: bootstrap 95% interval on raw Hit@10. The vertical axis starts at 0.25.**

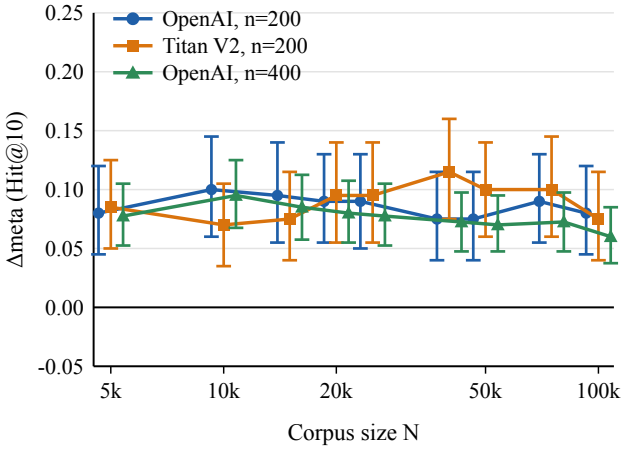


FIGURE 2. **Exact-search oracle lift in Hit@10 (`meta_in` minus raw) with paired bootstrap 95% intervals at each corpus size, for OpenAI (*n*=200 and *n*=400) and Titan V2 (*n*=200). The horizontal axis is logarithmic with the points at the nine ladder sizes, and the markers of the three series are offset horizontally for legibility.**

On the 21 multi-source questions alone the narrower first-type filter is slightly better than the all-types filter at 100k (0.905 against 0.857 Hit@10; raw 0.857; descriptive, n=21), because Hit@10 counts any gold document and a narrower filter removes more distractors. The first-type filter is worse on the other measures, because it excludes gold documents of the other listed types by construction: document recall@10 is 0.450 against 0.891 at 5k and 0.405 against 0.601 at 100k, and MRR 0.617 against 0.716 at 100k.

a: Absolute filtered quality still falls.

Oracle-filtered Hit@10 declines from 0.875 to 0.665 (OpenAI) over the ladder, the same 0.210 as the unfiltered condition (Table 4). Under round-robin, filtering shifts the curve upward by 0.075 to 0.100 up to 100k (0.06 to 0.115 across all arms) but does not slow the decline, consistent with crowding recurring inside each source-type partition. Dashboards that alert only on Δ_{meta} would miss this degradation. The v1 threshold $N^\star$ (Appendix A) depends on where a lift with no detected trend sits relative to τ=0.10: it is 15k at n=200 and 5k at n=400 with the same rule, so we no longer report it as a finding.

D. STRATIFIED DRIFT BY QUESTION TYPE

Point estimates differ across `question_type`: among strata with $n \geq 20$, `semantic` questions start weaker and fall hardest (Hit@10 0.622→0.311), `metadata` questions fall from 0.718 to 0.436, and `basic` remains comparatively robust (0.825→0.619). Smaller strata are reported only in the artifacts. With 45 and 39 questions in the semantic and metadata strata the differences in drop from the basic stratum (0.105 and 0.076) are not significant (their 95% intervals include zero), so this is a descriptive observation that motivates stratified monitoring, not evidence of a type effect.

E. INTEGRITY ABLATION (EVAL-ID COLLISION)

Before unique `question_id::question_type` keys, duplicate raw IDs across question types produced opposing Hit@10 flags on the same retrieval list and inflated empty-retrieval and membership mismatches; an exploratory repair that treated empties as quarantine also depressed raw Hit@10. After (i) unique eval IDs and (ii) Hit@k redefined as gold$\in$top-k membership, the integrity-clean v1 sweep reported 0 empty cells and 0 membership mismatches, and all results of this revision are built on those definitions together with exact search and per-question list invariants (Appendix B). Published claims use only the clean runs.

F. APPROXIMATE SEARCH AND THE QUERY PLANNER

a: v1 HNSW versus exact.

The v1 release reported its curves from one HNSW index queried with ef_search=200. We compare those numbers with exact search. For OpenAI the two agree at 5k (0.795), but at 50k and 100k the HNSW Hit@10 is 0.085 and 0.075 lower than exact (0.555 vs. 0.640 and 0.510 vs. 0.585), with 17 and 15 of 200 questions differing in their Hit@10 flag. For Titan the published "HNSW" lists are identical to the exact lists for all 200 questions at all nine sizes, in both the raw and the first-type-filtered condition (3,600 lists), so the v1 Titan arm was in effect exact search throughout. A replay of the v1 query sequence on the original copy (artifact `titan_v1_plan_replay`) shows why, even though that copy plans an individual Titan query at 75k or 100k with the index: the v1 Titan script sent the size bound as a query parameter through a driver (psycopg 3) that prepares a statement after five executions, and from 40k, where the bound no longer fits a 16-bit integer and a new statement is prepared, PostgreSQL's plan cache switched to a generic plan, which EXPLAIN shows as a B-tree scan on the size column with an exact sort (the plan built for the actual values uses the HNSW index), for every query at 75k and 100k. The replay reproduces exact lists at all nine sizes, whereas the same queries planned with

TABLE 5. Metadata lift (meta_in−raw) with paired 95% bootstrap CIs; the text quotes the 90% intervals of the summary rows from Table 13.

N	OpenAI n=200	Titan n=200	OpenAI n=400
5k	0.080 [0.045, 0.120]	0.085 [0.050, 0.125]	0.0775 [0.0525, 0.105]
10k	0.100 [0.060, 0.145]	0.070 [0.035, 0.105]	0.095 [0.0675, 0.125]
15k	0.095 [0.055, 0.140]	0.075 [0.040, 0.115]	0.085 [0.0575, 0.1125]
20k	0.090 [0.055, 0.130]	0.095 [0.055, 0.140]	0.080 [0.055, 0.1075]
25k	0.090 [0.050, 0.130]	0.095 [0.055, 0.140]	0.0775 [0.0525, 0.105]
40k	0.075 [0.040, 0.115]	0.115 [0.075, 0.160]	0.0725 [0.0475, 0.0975]
50k	0.075 [0.040, 0.115]	0.100 [0.060, 0.140]	0.070 [0.0475, 0.095]
75k	0.090 [0.055, 0.130]	0.100 [0.060, 0.145]	0.0725 [0.0475, 0.098]
100k	0.080 [0.045, 0.120]	0.075 [0.040, 0.115]	0.060 [0.0375, 0.085]

OpenAI n=200, 95% intervals: Δlift(100k−5k)=0.000 [−0.050, 0.050]; slope/tenfold=−0.008 [−0.048, 0.032]
Titan n=200, 95% intervals: Δlift(100k−5k)=−0.010 [−0.065, 0.040]; slope/tenfold=0.013 [−0.031, 0.057]
OpenAI n=400, 95% intervals: Δlift(100k−5k)=−0.0175 [−0.050, 0.015]; slope/tenfold=−0.017 [−0.045, 0.010]
Legacy meta_first lift (meta−raw): OpenAI 5k 0.080, OpenAI 100k 0.085, Titan 5k 0.085, Titan 100k 0.080.

their values use the index at 75k and 100k and match exact search for only 176 and 178 of 200 raw questions; the v1 comparison of the two embedders therefore compared a partly approximate OpenAI sweep with an exact Titan sweep. Part of the published OpenAI decline and of the v1 lift at 100k (+0.130 against +0.080 under exact search; Sec. V-C) were thus properties of the approximate index.

b: Which search ran?

We recorded the PostgreSQL execution plan (EXPLAIN (ANALYZE, BUFFERS)) for every embedder, condition, scale, and ef_search $\in \{200, 400, 800\}$ on two databases (Appendix E; all cells in the artifacts plan_audit_5433 and plan_audit_5434; all 200 questions at ef_search=200, a 40-question sample at 400 and 800), and the planner's choice alone (plain EXPLAIN) for all 200 questions in every cell, including every per-size table and ef_search $\in \{40, 100, 200, 400, 800\}$ (artifact plan_coverage_full; it gives the same shares as the ANALYZE audit at ef_search=200 and in every cell where all queries take the same plan, and in the eight filtered cells at ef_search=800 with mixed plans the 40-question sample differs from the full share by at most 0.05): a frozen copy of the original database and a fresh restore of the same dumps. The planner decides per statement between an HNSW index scan and an exact scan, and its decision is not determined by the ef_search setting alone; it also depends on table statistics and database state, which differ between the two databases: the original table's heap, enlarged by in-place updates, has 13,071 pages against 7,360 on the fresh restore, which raises the estimated cost of an exact scan (see the ablation below). The two databases also differ in memory settings (shared_buffers 128 MB against 2 GB, work_mem 4 MB against 64 MB). On the original copy, OpenAI raw queries used the index from 10k up at ef_search=200 but never at 5k, and not at all at 400 or 800 at any size (the planner chose exact scans). On the fresh restore the same setting ef_search=200 used exact scans for OpenAI raw queries up to 50k and the index only at 75k and 100k, while ef_search=800 used the index again at 10k and above. A setting such as "HNSW with ef_search=200" therefore does not say whether an approximate or an exact search ran, and the v1 ladder was a mixture of the two: exact at 5k, approximate from 10k. This inflated the early step between those sizes by about 0.075 Hit@10.

c: The ef_search *ablation.*

The v1 release reported that raising ef_search from 200 to 400 at 100k recovers 0.075 Hit@10 (0.510→0.585). The artifact search_mode_diagnosis_5434 and Appendix E show what happened on the original copy: while the planner used the index (ef 100 to 300), recall@10 against exact search was 0.893, 0.946, and 0.967 and Hit@10 was 0.495, 0.510, and 0.530; at ef_search $\geq$ 400 it switched to an exact scan, whose lists are identical to the exact reference and take about 44 s per 200 queries instead of about 1.5 s. The "recovery" was the switch to exact search, not extra graph traversal. Forcing the index at the higher settings (with plan assertions) recovers only part of the gap. The switch is cost-based: the estimated cost of the index plan at 100k rises with ef_search (median over the 200 queries 9,838 at 200 and 17,477 at 400; artifact search_mode_diagnosis_5434) and crosses that of the exact plan (15,785); on the fresh restore the index estimate rises the same way up to ef_search=750 (29,199) and then drops to 7,461 at 800, below the exact plan (10,110), so the planner returns to the index. The drop follows from pgvector 0.8.6's cost function, which subtracts a discount for vectors stored out of line (TOAST) once the estimated share of index pages read exceeds the table's heap pages: on the fresh restore (7,360 heap pages) that happens between 750 and 800, whereas the original copy's heap, enlarged by in-place updates to 13,071 pages, is not crossed below the maximum ef_search of 1,000 (artifact planner_cost_probe). The same page counts make the exact plan cheaper on the fresh restore (10,110 against 15,785), which is why its index is abandoned already at ef_search=300.

d: Further sources of approximate-search variation.

(i) With `hnsw.iterative_scan=relaxed_order`, 0.5% to 2.5% of the questions had neighboring-size filtered lists whose members appear in swapped order on the original copy (9 question-pair swaps in total; the first-type filtered lists also violate raw-to-filter containment, that is, a raw top-10 document of the filtered type is missing from the filtered top 10, in 56 of 1,800 filtered question-size lists: 16, 14, 12, 12, 1 and 1 at 10k, 15k, 20k, 25k, 40k and 50k and none at the other sizes, while no exact list does; the question identifiers of both kinds are in the artifact `v1_list_violations_openai`), versus none under `strict_order`. (ii) Two indexes built from the same dump differ slightly, so HNSW results are build-dependent (at 100k, Hit@10 0.510 and 0.540), whereas exact search is identical. The same holds for repeated sweeps of the original database itself: two `ef_search`=200 runs at 100k made on different dates (the first and the ablation sweep) have equal Hit@10 (0.510) but differ in Hit@1 (0.305 against 0.300), Hit@5 (0.460 against 0.465), and MRR (0.365 against 0.364), consistent with this build- and state-dependence of a table that had received in-place updates. (iii) Exact lists contain no score ties at ranks ten and eleven, so tie-breaking cannot change a Hit@10 flag.

G. DEPLOYED INDEX OF SIZE N

The single-index configuration above queries one graph over all documents, so it never measures an index that holds only N documents. For each embedder and each ladder size we therefore built a separate table with exactly the first N documents and its own HNSW index (m=16, ef_construction=64), queried it without any size predicate, and compared every result list with the exact reference list of the same question (Table 6). Three timed passes per cell gave identical metrics, as expected for a fixed index.

a: Approximation loss tends to grow with N.

For OpenAI at ef_search=200, recall@10 against exact search falls from 0.996 at 5k through 0.979 at 25k and 0.967 at 50k to 0.958 at 100k, and the Hit@10 loss relative to exact search rises from 0.005 to 0.025, 0.045, and 0.050. Raising ef_search to 800 restores recall to 0.9995 at 5k (Table 6) but only to 0.986 at 100k (loss 0.025), while lowering it to 40 costs 0.065–0.140. Approximation therefore adds to the drift: with ef_search=200 the Hit@10 decline from 5k to 100k is about 0.255 instead of the exact-search 0.210, and about 0.235 at ef_search=800. Four further builds per size with shuffled insertion order (`hnsw_build_replicates_openai`) reproduce these values: the mean Hit@10 loss rises from 0.008 at 5k to 0.046 at 100k (standard deviation 0.003 to 0.010; 0.020 at 20k and 0.039 at 75k break strict monotonicity), and recall@10 falls from 0.996 to 0.958 (standard deviation about 0.005). A rebuild of one replicate per size confirmed by `EXPLAIN`, for the probe query of every cell, an HNSW index scan (artifact `hnsw_replicate_plan_check`); the rebuilt values differ from the stored ones by at most 0.010 in Hit@10 and 0.010 in recall@10, as expected from non-deterministic builds. Titan's index is more accurate at the same settings (recall 0.999 at 5k and 0.975 at 100k, Hit@10 loss at most 0.015), so the size of this effect depends on the embedding model as well as on the index parameters.

b: Relation to the single-index result.

The v1 configuration lost 0.070 Hit@10 already at 5k (forced onto the index), against 0.005 for a dedicated index, and 0.085 against 0.045 at 50k; filtering a large graph down to a small corpus costs recall that a deployed index of that size would not lose. At 100k, where the filter removes only the 3,035 OpenAI rows beyond the ladder (2.9% of the graph), the single index loses 0.075 and the dedicated index 0.050 (the difference also reflects separate builds), which we do not resolve further because the difference is of the order of the build-to-build gap.

c: Metadata-filtered queries.

With a source-type filter the planner answers selective queries with the source-type B-tree and an exact sort and uses the HNSW index when the filter is less selective, so the plan depends on the question. The plan stored with each cell is that of its first question (Confluence, about 5% of the corpus), always an exact scan; a separate `EXPLAIN` showed single-type filters using the B-tree up to 50k and, at 100k, the index for the three largest sources. Filtered per-size results therefore mix exact and approximate scans per question and are not interpreted as approximate-search losses; the headline filtered numbers are exact.

d: Latency and capacity.

Appendix E and the artifacts `per_n_index_build_*` give table and index size, build time, and per-query latency. The HNSW index grows linearly, from 41 MB at 5k to 819 MB at 100k for OpenAI (d=1536), with build times of 2 to 56 s over five builds per size (OpenAI; the 100k builds, 17 to 30 s, are faster than those at 75k because PostgreSQL uses parallel build workers only once the table heap reaches 8 MB, which here happens between 75k and 100k; Table 16). For memory, we restarted the database before each size so that its buffer cache started empty, ran the 200 questions twice at ef_search=200, and counted the pages held in PostgreSQL's shared buffers (artifact `memory_footprint_per_n`): the queries brought 85% (100k) to 100% (5k) of the index into memory, 695 MB of 819 MB at 100k for OpenAI and 749 MB for Titan, plus 11 to 28 MB of table and out-of-line vector pages. The working set is therefore close to the whole index, and the index size is the memory to plan for if queries are to be served from memory. Each per-size table was built from scratch with one index build and `VACUUM ANALYZE` afterwards, so no `REINDEX` was needed. Median query latency at ef_search=200 rises only from 3.2 ms to 4.3 ms over a twenty-fold larger corpus (6.2 ms at the 95th percentile) and to about 11 ms at ef_search=800; exact scans cost 22 ms at 5k and 40 ms at 10k. These timings are indicative only (shared host, one client, warm caches). Even on a dedicated

table the planner sometimes chose a sequential scan (at 5k for ef_search $\geq$ 400, and at 10k for 800); such cells, marked † in Table 6, show the forced-index result and are not planner-selected behavior.

H. SPARSE CONTROL: BM25 ON THE TEXT THE DENSE MODEL SAW

Table 7 reports Okapi BM25 [30] (`rank_bm25.BM25Okapi`, k_1=1.5, b=0.75, tokenizer [a-z0-9]+, index rebuilt at every scale) on the same primary-200 bank and gold-pinned ladder. The v1 release truncated BM25 text at 4k characters, a cut that the dense model never saw; BM25 text was also the title followed by a chunk text that already begins with the title, so the title was counted twice. A dense-to-sparse comparison is only like-for-like when both see the same text, so we add variants (Table 7): the v1 rule, a title-once version, cuts at 8,191 characters and at 8,191 tokens, the untruncated database text, and `dense_text`, which is exactly the string the OpenAI model embedded (Titan saw the same text except for its 24,000-character cap); the benchmark text before ingestion-time truncation (`raw_untruncated`) differs from it for only 11 documents.

The length of the text matters: the Hit@10 loss between 5k and 100k is 0.155 with the 4k-character cut and 0.115–0.125 once the full text is used. In the like-for-like `dense_text` variant BM25 is above exact dense retrieval at every ladder size (by 0.015 at 5k up to 0.105 at 75k for OpenAI, and by 0.025 to 0.085 for Titan; the paired intervals include zero up to 15k for OpenAI and up to 25k for Titan) and loses 0.125 between the endpoints against 0.210 for OpenAI and 0.175 for Titan. The difference in loss is 0.085 for OpenAI (paired 95% interval $[0.015, 0.155]$) and 0.050 for Titan ($[-0.020, 0.120]$), so for OpenAI the endpoint evidence that BM25 declines less has an interval excluding zero and for Titan it does not. This depends on the text BM25 sees: with 8,191-token text the OpenAI difference is 0.095 ($[0.030, 0.160]$), and with the 4,000-character text of the first version it is 0.055 ($[-0.015, 0.120]$). The slope evidence is weaker: dense-minus-BM25 slope of Hit@10 against $\ln N$ is -0.026 ($[-0.049, -0.002]$) for OpenAI on all nine points and -0.019 ($[-0.047, +0.009]$) without the 5k point, and -0.021 ($[-0.046, +0.005]$) for Titan. We therefore state RQ2 as follows: on this bank BM25 stays above both dense models at every size and declines less between the endpoints, with an interval excluding zero for OpenAI; the evidence about slope is marginal, and growth pressure is not unique to dense retrieval.

I. SECONDARY EMBEDDER (AMAZON TITAN V2)

All comparisons in this section use exact search for both embedders on the same primary-200 bank and gold pins; intervals are paired percentile bootstraps over the 200 eval IDs (B=10,000, seed 42), which agree closely with randomization and paired t tests in IR evaluation [31]. A Titan run on the n=400 bank was not performed: it would require query embeddings of the additional questions with Titan, which we did not generate for this revision.

a: The decline replicates.

Titan raw Hit@10 falls from 0.785 to 0.610 (drop 0.175, 95% interval $[0.125, 0.230]$), Hit@1 from 0.620 to 0.385, and MRR from 0.671 to 0.465 (Table 4, Sec. V-B); the relative losses (38%, 22%, and 31% for Hit@1, Hit@10, and MRR) are somewhat smaller than for OpenAI (43%, 26%, 38%). Under oracle filtering the Titan Hit@10 drop is 0.185 ($[0.135, 0.240]$).

b: Are the two declines equivalent?

A confidence interval that contains zero is not evidence of equality. We therefore state a margin δ for the difference in Hit@10 between embedders and call two quantities equivalent when the 90% paired bootstrap interval of their difference lies strictly inside $(-\delta, +\delta)$ (the two one-sided tests (TOST) argument; an interval ending exactly at $\pm\delta$ counts as not equivalent). The margins (primary δ=0.05, sensitivity δ=0.10; the latter is about half of the 0.175 Titan drop, so equivalence within $\pm$0.10 means only that the two declines differ by at most about 0.09) are operational choices of the author, not externally registered. The OpenAI-minus-Titan difference in the raw Hit@10 drop between 5k and 100k is $+0.035$ (90% interval $[-0.020, +0.090]$): equivalent at δ=0.10, not at 0.05. For the metadata-filtered condition it is $+0.025$ ($[-0.025, +0.075]$), with the same verdicts, and the raw level at 100k differs by -0.025 ($[-0.075, +0.020]$), again equivalent only at 0.10 (Appendix D). With 200 questions the 90% interval of a paired difference has a half-width of about 0.05, so equivalence within $\pm$0.05 can only be shown when the point estimate is near zero. The data therefore show that the two declines are of the same size to within ten points of Hit@10, but they cannot show equality to within five. We do not claim that Titan drifts less, nor that the two drift identically.

c: Where the curves differ.

OpenAI's early step (5k→10k Hit@10 -0.060, $[-0.095, -0.030]$) is larger than Titan's (-0.030, $[-0.055, -0.010]$). Failures largely overlap: at 100k, 64 questions miss under both embedders, 103 hit under both, 19 hit only under Titan, and 14 only under OpenAI.

d: Metadata lift is similar across embedders.

Titan's oracle lift lies between $+0.070$ and $+0.115$ at every size (Table 5); the OpenAI-minus-Titan difference in lift is equivalent within $\pm$0.10 at all nine sizes and within $\pm$0.05 at four.

e: Input truncation.

The Titan arm truncated 90 documents at a 24,000-character cap (among them 5 of the 627 gold anchors), the OpenAI arm 11 documents at 8,191 tokens. Only two primary ques-

TABLE 6. Per-size HNSW approximate search (raw, `relaxed_order`): list recall@10 vs exact and Hit@10 loss.

Embedder	N	`ef_search`: recall / Hit@10 loss				
		40	100	200	400	800
OpenAI	5k	0.950 / 0.065	0.988 / 0.020	0.996 / 0.005	0.9995 / 0.000†	0.9995 / 0.000†
OpenAI	10k	0.926 / 0.075	0.972 / 0.035	0.990 / 0.025	0.998 / 0.005	1.000 / 0.000†
OpenAI	15k	0.922 / 0.065	0.973 / 0.035	0.988 / 0.020	0.998 / 0.010	0.9995 / 0.000
OpenAI	20k	0.901 / 0.085	0.957 / 0.050	0.983 / 0.025	0.996 / 0.005	0.999 / 0.000
OpenAI	25k	0.896 / 0.105	0.951 / 0.055	0.979 / 0.025	0.993 / 0.015	0.999 / 0.000
OpenAI	40k	0.870 / 0.080	0.943 / 0.055	0.978 / 0.030	0.994 / 0.010	0.9995 / 0.000
OpenAI	50k	0.854 / 0.075	0.940 / 0.055	0.967 / 0.045	0.981 / 0.025	0.994 / 0.010
OpenAI	75k	0.839 / 0.105	0.915 / 0.065	0.960 / 0.040	0.985 / 0.020	0.995 / 0.005
OpenAI	100k	0.819 / 0.140	0.913 / 0.085	0.958 / 0.050	0.974 / 0.040	0.986 / 0.025
Titan	5k	0.972 / 0.040	0.994 / 0.010	0.999 / 0.000	0.9995 / 0.000†	1.000 / 0.000†
Titan	10k	0.956 / 0.050	0.988 / 0.015	0.999 / 0.005	1.000 / 0.000	1.000 / 0.000†
Titan	15k	0.943 / 0.055	0.987 / 0.015	0.995 / 0.015	0.999 / 0.000	1.000 / 0.000
Titan	20k	0.942 / 0.040	0.985 / 0.010	0.995 / 0.010	0.999 / 0.005	1.000 / 0.000
Titan	25k	0.941 / 0.055	0.983 / 0.030	0.994 / 0.010	0.998 / 0.005	1.000 / 0.000
Titan	40k	0.920 / 0.055	0.981 / 0.025	0.994 / 0.015	0.998 / 0.010	0.9995 / 0.005
Titan	50k	0.928 / 0.040	0.978 / 0.020	0.990 / 0.015	0.998 / 0.010	0.9995 / 0.005
Titan	75k	0.897 / 0.060	0.966 / 0.015	0.983 / 0.015	0.994 / 0.010	0.998 / 0.010
Titan	100k	0.891 / 0.080	0.954 / 0.035	0.975 / 0.015	0.993 / 0.015	0.998 / 0.000

†Planner chose a sequential scan; recall and Hit@10 loss shown from the matching `seqscan_off` cell (6 cells). Recall between 0.9995 and 1 is printed to four decimals.

TABLE 7. Lexical vs dense Hit@10 across scales (primary-200). The last two rows give the 5k→100k drop difference (dense minus BM25) and its paired bootstrap 95% interval; BM25 variants marked (title+chunk) count the title twice.

Method (Hit@10)	5k	50k	100k	Drop 5k→100k
Dense exact OpenAI	0.795	0.640	0.585	0.210
Dense exact Titan	0.785	0.645	0.610	0.175
`chars4000` (title+chunk)	0.855	0.745	0.700	0.155
`dense_text_chars4000`	0.845	0.735	0.690	0.155
`chars8191` (title+chunk)	0.825	0.715	0.705	0.120
`tokens8191` (title+chunk)	0.820	0.725	0.705	0.115
`untruncated` (title+chunk)	0.820	0.725	0.705	0.115
`dense_text`	0.810	0.720	0.685	0.125
`raw_untruncated`	0.810	0.720	0.685	0.125
Drop difference 5k→100k, dense minus BM25 (`dense_text`): estimate and paired 95% interval				
OpenAI	0.085	$[0.015, 0.155]$		
Titan	0.050	$[-0.020, 0.120]$		

tions have a gold document that Titan saw truncated; excluding them changes Hit@10 by at most 0.008 (OpenAI) and 0.003 (Titan) at 5k, 50k, and 100k and the paired embedder comparisons by at most 0.005. To remove the difference altogether we re-embedded these 90 documents with the OpenAI model using, byte for byte, the 24,000-character text that Titan embedded (for every other ladder document the two inputs were already identical; artifact `matched_truncation_openai_primary200`), and re-ran the exact OpenAI ladder with the 90 vectors replaced; merging the same lists with the original vectors reproduces every published list, which checks the procedure. With identical inputs for both embedders, raw Hit@10 falls 0.800→0.585 (drop 0.215, $[0.160, 0.275]$, against 0.210), only three question–size Hit@10 flags change (all raw), filtered Hit@10 is unchanged at every size, and the OpenAI-minus-Titan difference in the raw drop is $+0.040$ (90% interval $[-0.015, +0.095]$, against $+0.035$), with the same equivalence verdicts. The embedder comparison therefore does not depend on the truncation rule.

J. ROBUSTNESS: LARGER EVAL BANK AND DISTRACTOR SCHEDULE

a: Larger stratified bank (n=400).

Extending the primary bank with 200 further stratified questions (seed 42; the primary 200 are a subset) and rerunning the exact ladder gives raw Hit@10 0.7725→0.5675 between 5k and 100k (drop 0.2050, paired 95% interval $[0.1675, 0.245]$) and oracle-filtered 0.8500→0.6275 (Table 4; this bank has granularity 0.0025, so its values are given to four decimals here). The primary 200 questions return identical top-ten lists in all 7,200 question-by-size-by-condition cells ($200 \times 9 \times 4$ conditions), so the lower level reflects harder

additional questions. *Correction.* The v1 release reported 0.710→0.468 for this bank; that figure was wrong because for 40 metadata questions the cached query vector belonged to a different question (the cache was keyed by a question identifier shared between question types). We re-embedded the 200 additional questions, keyed the lookup by identifier and type, checked the cached vectors of the primary questions against fresh embeddings (cosine 0.9968 to 1.0, against at most 0.724 between different questions; artifacts `query_vector_provenance_openai` and `query_vector_pairwise_openai`), fingerprinted all 400 vectors, and re-ran the bank. The bank is not an independent replication, and the 200 additional questions alone show a smaller lift (+0.075 at 5k, +0.040 at 100k; raw Hit@10 0.750→0.550), which is why the pooled lift drifts from 0.0775 to 0.0600 (Table 5).

b: Alternate distractor schedules.

The round-robin schedule changes the source mixture along the ladder (Sec. IV-A). We re-ran the exact ladder with the same questions and the same 99,998 embedded documents under three further orderings: uniform-random permutations of the distractors (seeds 42 and 0) and round-robin with seed 0. All are nested, give 0 list-invariant violations, and share the 100k corpus (raw 0.585, first-type-filtered 0.670), so they vary the path through source compositions, not the documents. Raw Hit@10 falls 0.790→0.585, 0.795→0.585, and 0.785→0.585 (drops 0.205, 0.210, 0.200, against 0.210 for the original schedule); the log-linear slope is -0.073 per $\ln N$ (uniform, seed 42) against -0.068; under the composition-proportional schedules it is -0.076 and -0.077 (seeds 0 and 42; -0.084 and -0.085 when the 150k and full-pool points are included, against -0.073 for round-robin over the same eleven points; these slopes were not tested for a difference); the metadata lift stays between 0.07 and 0.10. Titan shows the same pattern (raw 0.805→0.610, 0.795→0.610, 0.795→0.610, drops 0.195, 0.185, and 0.185 against 0.175 for round-robin; lifts 0.05 to 0.115). The order of arrival therefore does not drive the OpenAI decline or the lift between these endpoints (paired intervals of the drop difference lie within $[-0.030, +0.025]$); for Titan the uniform ordering with seed 42 increases the raw drop by 0.020 (95% interval $[0.005, 0.040]$) and the filtered drop by 0.035 ($[0.010, 0.060]$), a small effect on a decline that remains. It matters only where a filter source is exhausted early: for the 41 primary questions whose first listed source type is Confluence, Hit@10 under the first-listed-type filter stays at 0.829 from 40k to 100k under both round-robin orderings (Confluence is exhausted at about 46k; under the all-types filter the same questions give 0.805 from 40k to 75k and 0.780 at 100k) but keeps falling under the uniform ordering with seed 42 (0.878, 0.854, 0.829 at 50k, 75k, 100k).

c: Full pool and composition-proportional schedules.

The orderings above reorder the same 99,998 embedded documents. We therefore embedded the remaining 84,965 distinct extracted documents with the same model (no document exceeds the 8,191-token input limit, so none is truncated), which gives a union pool of 184,963 documents, and re-ran the exact ladder on the primary 200 questions over it (Table 8): round-robin extended to the full pool, and composition-proportional schedules in which the 627 gold anchors are pinned and the distractors form a uniformly random ordering of the pool (ten seeds; two shown), so their 100k corpora differ from the original one. A further schedule pins only the 272 gold documents of the evaluated bank and treats the other benchmark gold documents as ordinary distractors. All give 0 list-invariant violations, and round-robin reproduces the original ladder through 100k. Raw Hit@10 falls between 5k and 100k by 0.210 (round-robin) and by 0.232 on average over the ten proportional seeds (standard deviation 0.008, range 0.225 to 0.250), by 0.240 when only the evaluated gold is pinned (pinning all 627 anchors keeps the ladder identical across question banks and makes the headline drop smaller, not larger), and reaches 0.515 at the full pool in every schedule, where the corpora coincide. The decline is therefore similar across the schedules and pinning choices tested; the common full-pool endpoint is expected by construction, and all schedules draw from one downloaded pool, so this is robustness within the pool, not a confidence interval for enterprise growth. *The metadata lift is not constant beyond* 100*k.* Under round-robin it is 0.080 at 5k and 100k and 0.130 at the full pool; under the proportional schedules it ranges from 0.060 to 0.095 at 5k and from 0.085 to 0.115 at 100k and is 0.130 at the full pool. The full-pool value is a single corpus shared by all schedules, so it is one measurement, not a replication. Tested with paired intervals (artifact `lift_beyond_100k_openai_primary200`), the rise from 100k to the full pool is +0.050 ($[0.015, 0.085]$) under round-robin and +0.015 to +0.045 under the ten proportional seeds (one of the ten intervals excludes zero and two more reach it). The rise from 5k is +0.035 to +0.070 (one interval excludes zero and two more reach it). The direction is therefore consistent but the size is uncertain. Under round-robin the source of the rise is visible: beyond 100k only the five large sources still grow, so for the 100 questions on small sources filtered Hit@10 stays at 0.770 while unfiltered Hit@10 falls 0.680→0.630, and for the 86 questions on large sources filtered Hit@10 falls less (0.523→0.488) than unfiltered (0.442→0.349). The filter's advantage thus grows when the arriving distractors lie mostly outside the question's partition. The absence of a detected trend in the lift therefore holds for round-robin up to 100k only. Titan was not run on the full pool.

d: Drift by source group.

As a post-hoc, exploratory check we asked whether the headline drop is homogeneous across source groups. Grouping the 200 primary questions by their listed source types, the 100 questions that concern only the four smallest sources (Confluence, Jira, GitHub, Fireflies) fall from 0.840 to 0.680 (0.160) in raw Hit@10 and from 0.920 to 0.770 (0.150) with the oracle filter; the 86 that concern only large sources fall from 0.721 to 0.442 (0.279) raw and from 0.802 to 0.523 (0.279)

filtered; the remaining 14 mix both kinds. Under round-robin the difference between the two groups' raw drops is 0.119 (bootstrap 95% interval over questions within each group, [0.001, 0.240]), but it is 0.096 ([−0.022, 0.214]) under round-robin with seed 0, 0.054 and 0.064 under the two uniform orderings, 0.089 to 0.144 under the proportional schedules (most intervals include zero), and −0.027 to −0.047 for Titan (−0.029, [−0.137, 0.076] under round-robin). The decline is therefore present in both groups, but a dependence of its size on the source group is not established, the intervals are question bootstraps that are not clustered by benchmark identifier, and we do not explain it. Both groups show the rise of the lift beyond 100k (small 0.090 to 0.140, large 0.081 to 0.140); as described above, for the small-source group it comes entirely from the unfiltered side, because their filter partitions no longer grow.

e: Chunked-document control.

Everything above embeds whole documents. We re-embedded the 99,998 ladder documents as overlapping chunks with the same model (recursive splitter, title inside each 512-token chunk, 64-token overlap, a final piece under 64 tokens merged back; 381,847 chunks, of which 2.1% exceed 512 tokens, by at most 62). A document scores its best chunk (exact cosine, ties by identifier), and its chunks enter at its ladder position. Raw Hit@10 falls 0.830→0.625 between 5k and 100k (drop 0.205, [0.150, 0.260]) against 0.210 for whole documents (difference −0.005, [−0.070, +0.060], so no change is detected, although differences from −0.07 to +0.06 remain compatible with the data); the oracle filter gives 0.895→0.700, a lift of +0.075 at 100k. Chunks of 256 tokens with 32 overlap (first 25k) drop 0.100 against 0.125 (difference −0.025, [−0.080, +0.030]), and chunks without overlap (first 10k) 0.045 against 0.060. These figures use exact max-chunk document aggregation at six sizes (5k to 100k). Chunked pipelines usually pass the top ten *chunks* instead, and long documents can then fill several slots; scoring a question as a hit when any of its top ten chunks belongs to a gold document (artifact `chunk_topchunks_openai_primary200`), Hit@10 falls 0.805→0.615 (drop 0.190, [0.140, 0.245]; difference from whole documents −0.020, [−0.085, +0.045]) while the ten chunks cover on average 6.8 distinct documents at 5k and 8.0 at 100k, and with the oracle filter 0.885→0.690. The same code reproduces every published whole-document top-ten rank (12 cells), and the chunk embeddings passed alignment (re-embedded samples), duplicate, and norm checks.

K. HYBRID FUSION OF DENSE AND SPARSE RETRIEVAL

Because BM25 holds up better than dense retrieval on this bank (Sec. V-H), a natural remedy is to fuse the two rankings. We use reciprocal rank fusion (RRF) [32], $\mathrm{RRF}(d) = \sum_s 1/(k+\mathrm{rank}_s(d))$ with k=60 over the exact dense top-100 and the BM25 top-100 (on the text the dense model saw), ties broken by identifier. Table 9 shows that fusion raises Hit@10 above dense retrieval at every size by 0.11 to 0.15 (all paired 95% intervals exclude zero; 0.905 against 0.795 at 5k and 0.720 against 0.585 at 100k) and above BM25 at all sizes except 100k, where the gain is +0.035 ([−0.005, +0.075]). The level shift is larger than that of the oracle metadata filter (+0.08), but the drift remains: the 5k-to-100k drop is 0.185 for fusion against 0.210 for dense retrieval and 0.125 for BM25, and the difference between the fusion and dense drops in Hit@10 is not distinguishable from zero (−0.025, [−0.095, +0.045]), while for MRR@10 fusion's drop is smaller than dense retrieval's by 0.064 ([0.009, 0.119] for dense minus fusion). Fusion with the oracle metadata filter was not evaluated because the BM25 runs do not store filtered rankings.

a: How sensitive is fusion to its settings?

We repeated the fusion offline from the same stored top-100 lists with $k \in \{1, 10, 30, 60, 100, 200\}$ and a dense weight $w \in \{0.25, 0.4, 0.5, 0.6, 0.75\}$, scoring $2[w/(k + r_{\mathrm{dense}}) + (1-w)/(k + r_{\mathrm{BM25}})]$ (w=0.5 is standard RRF; artifact `rrf_sensitivity_openai_primary200`; the settings k=60 with $w \in \{0, 0.5, 1\}$ reproduce the published BM25, fusion, and dense values at every size). Table 10 shows that the constant k matters little: with equal weights fusion adds 0.08 to 0.155 Hit@10 over dense retrieval at every size for every k, and its 5k-to-100k drop stays between 0.175 and 0.185. The weight matters more: shifting weight toward the dense ranking (w=0.75) gives up most of the gain at 100k (0.610 to 0.660 against 0.710 to 0.720 at $k \geq 10$), while shifting it toward BM25 (w=0.25) costs less. No setting removes the decline: every fused drop lies between 0.165 and 0.230, and none differs from the dense drop with a paired interval excluding zero. Because the grid is evaluated on the test bank itself, the best cell is an optimistic choice rather than a tuned result. In practice this supports a simple rule on this benchmark: fuse dense and BM25 rankings with equal weight (any k between 10 and 200), do not weight the fusion toward the dense side as the corpus grows, and keep monitoring, since fusion raises the level but does not change the rate of decline.

L. WHEN THE FILTER IS WRONG: ROUTING ERRORS

The oracle filter assumes a router that always knows the right source types. To bound the cost of routing errors we simulate four failure modes with exact search: the router returns a single wrong source type with probability p (*wrong type*); it drops one of the true types, or abstains when only one exists (*drop gold*); it adds one extra type to the true set (*superset*); or it abstains and the system searches the unfiltered corpus (*abstain*). The wrong or extra type is drawn uniformly from the other types present in the corpus, and the dropped type uniformly from the true types; only 21 of the 200 primary questions have more than one true type, so drop gold equals abstain for the rest. The *wrong type* and *abstain* expectations also have closed forms, $(1-p)$ times the oracle value and $(1-p)$ times the oracle plus p times the raw value; the Monte Carlo columns scatter around them. The top ten of any filter set is the top ten of the union of the per-type top-

TABLE 8. Full-pool exact ladders (OpenAI, primary-200): raw / `meta_in` Hit@10 at selected sizes and the lift in Hit@10 at 5k, 100k, and the full pool (184,963 documents).

Schedule	5k	50k	100k	150k	Full	Δ_{5k}	Δ_{100k}	Δ_{full}
round-robin	0.795 / 0.875	0.640 / 0.715	0.585 / 0.665	0.540 / 0.650	0.515 / 0.645	+0.080	+0.080	+0.130
proportional, seed 42	0.825 / 0.900	0.655 / 0.755	0.595 / 0.680	0.520 / 0.660	0.515 / 0.645	+0.075	+0.085	+0.130
proportional, seed 0	0.800 / 0.875	0.650 / 0.740	0.575 / 0.685	0.520 / 0.660	0.515 / 0.645	+0.075	+0.110	+0.130
proportional, evaluated-bank gold pinned only	0.810 / 0.895	0.655 / 0.730	0.570 / 0.670	0.530 / 0.660	0.515 / 0.645	+0.085	+0.100	+0.130

TABLE 9. Dense, BM25 (`dense_text`), and RRF fusion (k=60) on primary-200; paired bootstrap 95% CIs. The lower block gives the 5k→100k drop in Hit@10 and in MRR@10 with intervals. Over all nine sizes, RRF minus dense Hit@10 ranges from +0.110 to +0.150. Intervals here come from this analysis's own bootstrap, which resamples the questions in a different order, and can differ by 0.005 from other artifacts for the same quantity (e.g., the BM25 drop interval is [0.080, 0.170] here and [0.080, 0.175] in `analysis_exact`).

N	Dense H@10	BM25 H@10	RRF H@10	RRF MRR	RRF−dense H@10	RRF−BM25 H@10
5k	0.795	0.810	0.905	0.770	0.110 [0.060, 0.160]	0.095 [0.050, 0.145]
25k	0.670	0.745	0.810	0.669	0.140 [0.085, 0.195]	0.065 [0.020, 0.115]
50k	0.640	0.720	0.770	0.623	0.130 [0.070, 0.190]	0.050 [0.005, 0.100]
100k	0.585	0.685	0.720	0.584	0.135 [0.080, 0.195]	0.035 [−0.005, 0.075]

Drop 5k→100k	Hit@10 [95% CI]	MRR@10 [95% CI]
Dense	0.210 [0.155, 0.270]	0.249 [0.204, 0.297]
BM25	0.125 [0.080, 0.170]	0.183 [0.141, 0.228]
RRF k=60	0.185 [0.130, 0.240]	0.185 [0.141, 0.231]
RRF−dense drop, Hit@10	−0.025 [−0.095, 0.045]	

TABLE 10. Weighted reciprocal rank fusion, $2[w/(k + r_{dense}) + (1 - w)/(k + r_{BM25})]$, on primary-200 (exact dense top-100 and BM25 `dense_text` top-100). Each cell gives Hit@10 at 100k / Hit@10 drop from 5k to 100k. Dense alone: 0.585 / 0.210; BM25 alone: 0.685 / 0.125. The grid is evaluated on the test bank itself (in-sample), and no fused setting has a drop whose paired 95% interval of the difference from the dense drop excludes zero.

Dense weight w	k=1	k=10	k=60	k=200
0.25	0.695 / 0.175	0.690 / 0.165	0.690 / 0.185	0.705 / 0.185
0.40	0.685 / 0.190	0.720 / 0.175	0.705 / 0.190	0.705 / 0.190
0.50	0.700 / 0.175	0.710 / 0.180	0.720 / 0.185	0.715 / 0.180
0.60	0.700 / 0.190	0.715 / 0.170	0.670 / 0.220	0.670 / 0.215
0.75	0.680 / 0.205	0.610 / 0.225	0.635 / 0.230	0.660 / 0.220

ten lists, which we verified against direct filtered queries on 50 random cases (no mismatch); each setting uses 200 Monte Carlo repetitions per question, and p=0 reproduces the oracle exactly.

Table 11 shows that the modes differ sharply. The first mode is a worst case by construction: a wrong type never contains a gold document, so Hit@10 equals $(1 - p)$ times the oracle value and the break-even is exactly $p^* = \Delta_{meta}/\text{Hit@10}_{oracle} = 0.091$ (5k), 0.105 (50k), and 0.120 (100k), and 0.091 to 0.131 over all nine sizes (largest at 75k); the Monte Carlo estimates of the simulation (0.091, 0.103, 0.118) only add noise to this formula. At p=0.2 and 100k Hit@10 is 0.531 (Monte Carlo; the closed form gives 0.532), below the unfiltered 0.585. This is a bound, not an estimate for a real router, whose errors are often partly right (a related source type), but it shows how little room for error the filter has. In contrast, errors that keep the gold type in the filter are benign: adding one extra type costs at most 0.022 Hit@10 at the sizes of Table 11 (0.029 across all nine sizes) even at p=1 (0.644 at 100k, still above unfiltered), abstaining degrades linearly to the unfiltered level, and dropping a true type remains above unfiltered until p is close to one. The last statement needs a caveat: only 21 of the 200 primary questions list more than one source type, so for the other 179 the only type is dropped and the router effectively abstains; on the 21 multi-source questions alone (a descriptive subset) dropping one true type changes Hit@10 by 0.003 at 100k (0.855 against 0.857 for the oracle set), because one remaining type usually still holds a gold document. A deployed router should therefore be tuned for the precision of the filter, preferring to widen or abstain rather than to commit to a wrong type; the oracle lift of this paper is an upper bound on what such a router can deliver.

M. ANSWERS TO RESEARCH QUESTIONS

a: RQ1.

Yes. Under exact search and gold-anchor pinning, Hit@1, Hit@10, and MRR all decline with $\log N$: for OpenAI, Hit@10 falls by 0.210 (95% interval [0.155, 0.270]) through 100k and to 0.515 at the full pool, Hit@1 by 43% and MRR by 38% of their starting values (Hit@10 26%), and DocRecall@10 falls 0.765→0.540. The decline is similar on the n=400 bank (0.205) and under every schedule tested (drops 0.200 to 0.250); an approximate index adds 0.005 to 0.05 at ef_search=200.

b: RQ2.

BM25 on the text the OpenAI model embedded has higher Hit@10 than both dense retrievers at every size (point esti-

TABLE 11. Simulated metadata routing errors (expected Hit@10; Monte Carlo over per-question oracle lists). Break-even *p*: the error rate at which expected Hit@10 falls to the unfiltered level, interpolated between *p*=0 and *p*=1 (expected Hit@10 is linear in *p*); *never*: not below the unfiltered level even at *p*=1. The artifact also stores an interpolation over the full *p* grid, which differs by at most 0.002.

Mode	*N*	Hit@10 at router error *p*							Break-even *p*
		0	0.05	0.1	0.2	0.3	0.5	1	
wrong_type	5k	0.875	0.832	0.787	0.699	0.614	0.440	0.000	0.091
wrong_type	50k	0.715	0.680	0.642	0.571	0.502	0.359	0.000	0.105
wrong_type	100k	0.665	0.632	0.597	0.531	0.468	0.333	0.000	0.120
wrong_dropgold	5k	0.875	0.871	0.868	0.860	0.851	0.837	0.800	never
wrong_dropgold	50k	0.715	0.711	0.709	0.700	0.694	0.680	0.645	never
wrong_dropgold	100k	0.665	0.661	0.657	0.649	0.641	0.625	0.585	0.996
superset	5k	0.875	0.874	0.873	0.871	0.869	0.866	0.856	never
superset	50k	0.715	0.714	0.713	0.711	0.709	0.704	0.693	never
superset	100k	0.665	0.664	0.663	0.661	0.659	0.655	0.644	never
abstain	5k	0.875	0.871	0.867	0.860	0.852	0.836	0.795	never
abstain	50k	0.715	0.712	0.708	0.700	0.693	0.678	0.640	never
abstain	100k	0.665	0.662	0.657	0.650	0.642	0.625	0.585	never

mates; BM25 on the text Titan embedded, with its 24,000-character cap, gives nearly the same curve, $0.810 \rightarrow 0.685$, identical except 0.695 against 0.700 at 75k) and loses 0.125 between the endpoints against 0.210 (OpenAI) and 0.175 (Titan); the interval of the difference in loss excludes zero for OpenAI (0.085, $[0.015, 0.155]$) but not for Titan (0.050, $[-0.020, 0.120]$). Reciprocal rank fusion lifts Hit@10 by 0.11–0.15 and still declines (0.185 against 0.210 for dense; the difference, -0.025 with interval $[-0.095, 0.045]$, is not detectably different from zero; Sec. V-K).

c: RQ3.

Against an exact baseline the oracle lift is about $+0.08$ (OpenAI) and $+0.07$ to $+0.115$ (Titan) at every size up to 100k under round-robin ($+0.05$ at one size of a uniform Titan ladder); we detect no trend up to 100k under round-robin, but the lift is 0.13 at the full pool and varies with the schedule and the bank (0.060 at 100k on the n=400 bank). It is an upper bound: a router whose errors exclude every gold document erases it at an error rate of about 9–13%, while errors that only widen the filter or abstain are benign (Sec. V-L).

d: RQ4.

Yes. Titan V2 declines by 0.175 ($[0.125, 0.230]$); the difference between the declines is $+0.035$ (90% interval $[-0.020, +0.090]$), equivalent within ± 0.10 but not shown equivalent within ± 0.05 (Sec. V-I). Two embedders do not establish universality.

e: RQ5.

Approximate search adds loss that grows with N (replicated over five builds per size, not strictly monotone) and depends on the embedder: 0.005 to 0.050 for OpenAI in the primary build (up to 0.060 across the five builds) and at most 0.015 for Titan with a dedicated HNSW index. PostgreSQL decides per statement between an index scan and an exact scan, so one ef_search value gave exact search on one database and HNSW on another, which explains an apparent $+0.075$ effect in v1 (Sec. V-F).

VI. DISCUSSION

Drift under pinned-gold growth is consistent with larger corpora increasing near-neighbor competition: distractors enter the top-k even though gold embeddings are unchanged. Two mechanisms are consistent with our curves: (i) semantic near-duplicates accumulate in the same embedding neighborhood, and a fixed-dimension embedding can represent only a limited set of top-k subsets [14]; (ii) approximate-search recall at fixed index parameters can fall as N grows. The whole decline survives exact search, so it is not caused by approximate search and is consistent with distractors displacing labeled gold documents (mechanism (i), not isolated by analyzing near-duplicates); the dedicated-index study shows that (ii) adds up to 0.05 Hit@10 at 100k for OpenAI and little for Titan. The decline appears under two embedding models and under four orderings of the same embedded documents. A caveat applies to the construct: Hit@k counts only the labeled gold documents, so displacing documents that are in fact relevant but unlabeled would also lower it; a stricter two-judge audit specified in advance rated none of the 420 replacement documents as fully answering the question and 417 to 419 as off-subject, while rating 80 to 98% of known gold documents as on-subject; label completeness is therefore bounded only weakly (Sec. VII).

a: What helps.

Three levers change the curves by different amounts. An oracle metadata filter adds about $+0.08$ Hit@10 and, up to 100k under round-robin, shifts the curve without slowing it ($+0.13$ at the full pool), so the decline continues inside every partition; its value hinges on the router, which must be right about 90% of the time or more when its mistakes name a wrong type. Hybrid reciprocal rank fusion adds $+0.11$ to $+0.15$, more than the oracle filter, and needs no labels, but it too declines with N. None of them removes the drift;

capacity monitoring on a frozen gold set remains necessary, with Hit@1 and MRR the most sensitive monitors.

b: Measuring through a vector index.

A quality-versus-size curve read from one index can mix search modes. Report whether an exact or an index scan produced each number with plan evidence, and measure approximation loss on a per-size index or against an exact reference.

c: Coverage versus drift.

Coverage-oriented corpus scaling [5] and pinned-gold drift answer different questions: more documents can raise end-to-end question answering (QA) when answers were missing and still lower labeled first-stage Hit@k when answers were already indexed, so dashboards should track both.

d: Chunking.

Chunking multiplies the vectors competing for each query (here $3.8\times$ at 100k documents) and long documents contribute more of them. Scoring a document by its best chunk lets a long gold document match a narrow question but also gives each long distractor more chances to enter the top ten [33]. The control in Sec. V-J shows no detectable change in the decline from these effects for the setting tested (one embedder, one splitter), both with max-chunk scoring and when the top ten chunks are retrieved directly; there, long documents take several slots (the ten chunks cover only 6.8 to 8.0 distinct documents on average), yet the drop (0.190) is close to the whole-document drop. Other splitters, summaries, or late interaction are untested.

e: Implications for practitioners.

(1) Track Hit@1/MRR on a frozen gold set stratified by question type, not only latency. (2) Treat metadata schemas as retrieval features, but measure the router's error rate: in the worst case about 9–13% cancels the filter's gain. (3) Consider hybrid fusion before more elaborate remedies, since it helped here (the benchmark's synthetic questions may favor lexical overlap): equal-weight RRF was robust to its constant k, weighting it toward the dense ranking lost most of the gain, and the fused curve still declines, so fusion complements but does not replace growth monitoring. (4) Keep gold-anchor pinning in offline evaluations. (5) Report the execution plan behind approximate-search numbers. These recommendations concern document-level labeled Hit@k on this benchmark extract, not chunked production systems.

VII. THREATS TO VALIDITY

a: Internal validity.

All headline numbers are exact-search numbers, so index construction and planner behavior cannot affect them. The oracle `source_type` filter upper-bounds production filters (Sec. V-L bounds the loss from wrong routing under four simplified error models, not under a learned router). Query vectors are cached and fingerprinted (Sec. V-J). Latencies come from a shared host and a single client and are indicative. Titan was not run on the n=400 bank because that would require 200 new Titan query embeddings, which we did not generate.

b: External validity.

EnterpriseRAG-Bench is a synthetic English benchmark; the results describe growth within a convenience pool of 184,966 extracted entries (184,963 distinct and embedded for OpenAI, 99,998 for Titan; Gmail and Slack capped at 40,000 each) and may differ for proprietary corpora with heavier near-duplicate rates, other languages, multimodal content, other question distributions, or other vector databases and index types. We test one HNSW configuration (m=16, ef_construction=64) in pgvector 0.8.6, two commercial single-vector embedders, no reranker, no multi-vector or sparse-learned retriever, whole-document vectors (the chunked control of Sec. V-J tests one splitter), an idealized oracle router rather than a learned one, and a fixed gold set whose documents are never removed or changed; real corpora also delete, update, and re-embed. Growth is simulated by adding distractors in a controlled order, not by replaying an ingestion history. The primary bank is skewed toward small sources: by first listed source type it holds 41 Confluence, 39 Jira, 26 Linear, 25 HubSpot, 25 Google Drive, 19 GitHub, 14 Fireflies, 7 Gmail, and 4 Slack questions, so the drift, lift and filter results may not carry over to banks with a different source mix; an exploratory split by source group (Sec. V-J) shows a larger drop for questions on large sources under round-robin (0.279 against 0.160) that ranges from 0.054 to 0.144 under the other OpenAI schedules (most intervals include zero) and reverses for Titan, so we make no claim about its cause. The oracle lift and the router analysis use exact search; filtered approximate search with post-filtering and iterative scans was not measured (Sec. V-F). Pinning all 627 gold anchors makes gold documents 12.5% of the 5k corpus and 0.6% of the 100k corpus, a dilution confound that the evaluated-gold-only schedule addresses only in part. The language-model judges are not human and missed some known-relevant documents (full or partial on 77% and 50% of positive controls, fully answering on 11% and 9%, although 98% and 80% were rated on-subject when conflict verdicts are counted; see below); neither judge of the second audit shares a vendor with the primary embedder.

c: Construct validity.

Hit@k and DocRecall measure labeled document retrieval, not grounded answer faithfulness, citation correctness, or multi-hop reasoning. L3 LLM relevance scores and L4 LLM-assisted Hit@10-fairness audits are secondary corroboration only. BM25 uses default parameters, not a tuned or field-weighted variant, and is compared on the text the dense model saw; the fusion results use k=60, and the sensitivity grid over k and the dense weight is evaluated in-sample, not tuned on held-out questions.

d: Label completeness.

Hit@k counts only labeled gold documents, so a displacing distractor that also answers the question would lower Hit@k without retrieval having failed. We audited this on the 42 primary questions whose gold document is in the top ten at 5k and not at 100k (raw, exact search). A first audit with one judge (OpenAI GPT-5.6 Terra, quote-required rubric, no reference) marked 38 of the 420 non-gold top-ten documents at 100k as fully answering, covering 13 of the 42 questions, which would have shrunk the drop from 0.210 to 0.145. We then re-audited all 420 documents with a stricter design fixed in advance (artifact `audit_v3`; the protocol file was written before any judge call and its hash is stored in every output, but it was not registered externally): two judges from other vendors (Google Gemini 4 and xAI Grok 4.7), the question's known-correct gold document as a reference, a required match of the question's subject (entity, record, date), a verbatim quote verified by code, and a random and a near-miss control in every call. Neither judge rated any of the 420 documents as fully answering (Gemini: 3 partial; Grok: 1 partial; none shared), none of the 13 earlier questions was confirmed, and for 36 of the 38 documents the first audit had marked as full, both judges recorded a different subject (37 for each judge alone; for example a different incident or record). All 140 random and near-miss controls were rejected (false-positive rate 0, 95% interval up to 0.03). The positive controls had to be second gold documents of questions with several gold documents, so all 44 come from such questions, 39 of them of types whose answer by design spans several documents (project-related 19, completeness 12) or sets documents against each other (conflicting information 8); 40 of the 42 audited questions, in contrast, have a single gold document. A second gold document of such a question usually holds only part of the answer or the opposing side of a conflict, and this is what the judges reported: they recognized the controls as the same subject (any verdict other than no) in 43 and 35 of 44 cases (98% and 80%), against 3 and 1 of the 420 replacement documents, but rated only 5 and 4 of the 44 as fully answering (34 and 22 as full or partial; 9 and 13 as conflict). The controls therefore show that the judges recognize the subject reliably but not how often they would miss a full answer, so missed answers cannot be excluded. The protocol's bound on the share of the decline due to incomplete labels (about 7% under the full-answer criterion) assumed a judge sensitivity of at least 0.9; because sensitivity was lower, we do not report that bound as a result (the protocol's sensitivity-rescaled value is zero only because no question was credited). Even if every replacement document judged to be on the same subject were credited, 2 (Gemini) or 1 (Grok) of the 42 questions would become hits, 0.010 or 0.005 of the drop of 0.210. The evidence is therefore the subject mismatch, and incomplete labels appear to explain only a small part of the decline. An earlier exploratory `gpt-4o` judge without a reference document or subject rule (artifact `llm_displacement_audit_gpt4o`) rated 10 of the 42 top displacing documents as answering and 10 as partial, but rated documents that were already in the top ten at 5k the same way (12 and 7; 4 against 3 discordant pairs), so this lenient judge did not single out the displacers. The audit is model-judged, judged 12 or 13 candidates per call, and is not human relabeling; the conclusions are about labeled Hit@k.

e: Statistical conclusion validity.

The primary bank has n=200 questions; intervals are paired percentile bootstraps over questions with the corpus held fixed, so they do not include the variation due to which distractors enter, and no correction for multiple comparisons is applied (interval statements are descriptive). The 200 primary questions have distinct texts and gold sets but 188 distinct benchmark identifiers (twelve occur under two question types); a bootstrap over these identifiers gives $[0.154, 0.266]$ for the primary drop, against $[0.155, 0.270]$. The n=400 bank contains the primary questions and is not an independent replication, and its additional 200 questions alone show a smaller lift. The equivalence margins (0.05 primary, 0.10 sensitivity) are operational choices, not externally registered; with 200 questions equivalence within 0.05 could not be shown for the embedder comparison, and a margin of 0.05 is large relative to a lift of about 0.08, so a decline of the lift of up to about half is not excluded by the statement that no trend in the lift is detected up to 100k. The threshold $N^\star$ is unstable and not used (Appendix A); small question-type strata have high variance.

VIII. REPRODUCIBILITY NOTES

Code, published metrics, and restorable pgvector dumps are hosted at https://pedapudibhargav.github.io/vector-drift-study/ (GitHub repository https://github.com/pedapudibhargav/vector-drift-study; the scripts, artifacts and generated tables of this revision are in release v2.0.4, and database dumps of the two embedding arms are on the GitHub release v1.0.0-artifacts, which restores the tables up to 100k documents; the full-pool and chunk embeddings are not released as dumps but can be regenerated by re-embedding with the published scripts), so reviewers can restore both embedding arms without re-paying for document embedding. The artifacts of this revision (`scripts/erb/revision1`, `artifacts/published/revision1`) comprise the exact ladders with per-question lists (OpenAI n=200 and n=400, Titan n=200, uniform-random and seed-0 ladders), the execution-plan audits and diagnosis on two databases, the per-size index build and evaluation, the BM25, fusion, routing, and equivalence analyses, the document-length statistics, the query-vector provenance record, and a validation harness with mutation tests and a claim registry that runs in continuous integration without database access. Every table of the paper is generated from these files by script, and a replay guide lists the commands in order.

IX. CONCLUSION

Measured with exact search on EnterpriseRAG-Bench, labeled dense retrieval quality declines as the corpus grows around pinned gold documents: Hit@10 falls by 0.210 for OpenAI `text-embedding-3-small` and by 0.175 for Amazon Titan V2 between 5k and 100k documents, rank-sensitive metrics lose proportionally more, and the two declines are equivalent within 0.10 but not provably within 0.05 at n=200. BM25 on the same text declines less, with a nominal interval excluding zero only for OpenAI. An oracle metadata filter adds about +0.08 against an exact baseline up to 100k on the primary bank (+0.05 to +0.115 across schedules and embedders; 0.060 at 100k on the n=400 bank; 0.13 at the full pool) and is erased, in a worst-case model, by routing errors that exclude every gold document at a rate of about one in ten; reciprocal rank fusion adds +0.11 to +0.15; neither removes the drift. Approximate search adds loss (0.005 to 0.050 Hit@10 at ef_search=200 for OpenAI in the primary build and up to 0.060 across five builds per size, growing with N but not strictly monotone), and, more importantly for measurement, the search mode behind an `ef_search` setting is decided by the planner and differs across databases, which can produce a spurious effect and a lossy baseline. A chunked-index control shows no detectable change in the decline, whether documents are scored by their best chunk (drop difference -0.005, 95% interval $[-0.070, 0.060]$, which does not establish equivalence within ± 0.05) or the top ten chunks are retrieved (-0.020, $[-0.085, 0.045]$), and giving both embedders byte-identical input text leaves the conclusions unchanged. Future work includes other chunking strategies, multi-vector late-interaction retrievers [34], open-weight embedders, real ingestion histories, and learned routers.

APPENDIX A LEGACY FILTER AND $N^\star$ SENSITIVITY

The v1 release restricted the oracle filter to the first listed source type of each question (`meta_first`); the revision uses all source types of the question (`meta_in`). The exact-search lift differs by at most 0.005 at every size of the round-robin ladders (0.010 at two sizes of one uniform ladder), and `meta_in` coincides with the filter built from the gold documents' own source types on the primary-200 bank and for 399 of the 400 questions of the larger bank. The sustained-collapse threshold $N^\star(\tau) = \min\{N : \Delta_{\text{meta}}(N') < \tau \; \forall N' \geq N\}$ of the v1 release, with τ=0.10, gives 15k (OpenAI, n=200), 5k (n=400), and 100k (Titan); because the lift shows no detected trend and lies near τ (Table 5), $N^\star$ is set by sampling noise and is not reported as a result.

APPENDIX B VALIDATION HARNESS

Every result file is checked by a harness that fails closed: schema and provenance of each file; list invariants on all exact lists (containment of filtered lists, nested-corpus consistency, score monotonicity; 0 violations); search-mode truth (a file claiming approximate search needs plan evidence, one claiming exact search must show no HNSW index scan; the exception is the HNSW build-replicate file, whose stored plan flag is a false negative of a name-based check and whose plan evidence is the rebuild check of Sec. V-G); query-vector provenance (each vector against its fingerprint and a fresh embedding of its text); an independent second implementation of the headline statistics; data hygiene; and a registry of the manuscript's headline claims against the artifacts. Deliberate corruptions (a swapped query vector, a metadata list equal to the raw list, a dropped top document, an exact file labeled approximate, a claim off by 0.01, among others) are each detected. The v1 release would have failed this harness: its approximate-search cells carried no plan evidence and its n=400 bank used the wrong vector for 40 questions.

APPENDIX C SECONDARY EMBEDDER LADDER (TITAN V2)

Table 12 gives the Titan primary-200 ladder under exact search at all nine sizes, raw and filtered, in the same form as Table 3 (artifact `exact_ladder_titan_primary200.json`; summary in Table 4). The protocol differs from the OpenAI arm in the embedding model (Bedrock `amazon.titan-embed-text-v2:0`, d=1024, unit normalization) and in the input cap (24,000 characters; 90 documents truncated, 5 of them gold anchors); the pins and the `scale_rank`$<N$ sets are identical (99,998 documents at 100k).

APPENDIX D EQUIVALENCE ANALYSIS

Table 13 lists the equivalence quantities reported in the text (all nine sizes of each are in the artifact `analysis_equivalence`) each with its 90% paired bootstrap interval and verdict at δ=0.05 and δ=0.10; "yes" means the interval lies strictly inside $(-\delta, +\delta)$ and "no" means it does not.

APPENDIX E SEARCH EXECUTION AND POOL DETAILS

Table 14 gives the share of queries whose execution plan used the HNSW index, on the frozen original database copy and on a fresh restore, for both embedders, the raw and the filtered condition, each size, and each `ef_search`. It shows the planner dependence for every arm. At ef_search=200 Titan raw used the index only at 75k and 100k on the original copy (only at 100k on the fresh restore) and ran exact scans at all smaller sizes. The v1 Titan lists nevertheless equal exact search at every size because the v1 sweep reused a generic prepared plan, an exact scan, at the larger sizes (Sec. V-F). Filtered queries used the index for only part of the questions, and on the original copy OpenAI filtered queries did so only from 40k up at ef_search=200. The two databases also differ in table statistics: on the original copy the main tables had last been analyzed by autovacuum (OpenAI on 27 September, Titan on 23 September 2026, with 6,890 Titan rows modified since), whereas the fresh restore and every per-size table were analyzed explicitly after loading (artifact `plan_coverage_full`). On the per-size tables the planner used the index for all 200 questions at every size for ef_search $\leq$ 200, and chose an exact scan only at 5k for 400 and at 5k and 10k for 800. Table 15 shows the planner behavior behind the

TABLE 12. Amazon Titan V2 exact search (primary-200): raw and `meta_in`.

Condition	N	Hit@1	Hit@5	Hit@10	Hit@10 95% CI	DocR@10	MRR
`raw`	5k	0.620	0.730	0.785	[0.725, 0.840]	0.766	0.671
`raw`	10k	0.595	0.695	0.755	[0.695, 0.815]	0.731	0.641
`raw`	15k	0.560	0.685	0.735	[0.675, 0.795]	0.710	0.612
`raw`	20k	0.525	0.645	0.700	[0.635, 0.765]	0.677	0.583
`raw`	25k	0.505	0.640	0.695	[0.630, 0.760]	0.670	0.566
`raw`	40k	0.455	0.620	0.650	[0.585, 0.715]	0.620	0.525
`raw`	50k	0.425	0.605	0.645	[0.580, 0.710]	0.615	0.504
`raw`	75k	0.400	0.595	0.615	[0.545, 0.685]	0.583	0.483
`raw`	100k	0.385	0.565	0.610	[0.540, 0.680]	0.577	0.465
`meta_in`	5k	0.700	0.825	0.870	[0.820, 0.915]	0.858	0.757
`meta_in`	10k	0.650	0.795	0.825	[0.770, 0.875]	0.805	0.710
`meta_in`	15k	0.610	0.775	0.810	[0.755, 0.865]	0.786	0.682
`meta_in`	20k	0.580	0.755	0.795	[0.740, 0.850]	0.770	0.650
`meta_in`	25k	0.570	0.745	0.790	[0.735, 0.845]	0.765	0.637
`meta_in`	40k	0.515	0.690	0.765	[0.705, 0.825]	0.733	0.594
`meta_in`	50k	0.500	0.655	0.745	[0.685, 0.805]	0.715	0.576
`meta_in`	75k	0.475	0.650	0.715	[0.650, 0.775]	0.684	0.554
`meta_in`	100k	0.470	0.635	0.685	[0.620, 0.750]	0.656	0.542

TABLE 13. Equivalence analysis: paired 90% bootstrap intervals; a quantity is called equivalent when its interval lies strictly inside $(-\delta, +\delta)$.

Quantity	Est.	90% CI	Equivalent, δ=0.05	Equivalent, δ=0.10
Raw Hit@10 drop 5k→100k, OpenAI − Titan	0.035	[−0.020, 0.090]	no	yes
Filtered Hit@10 drop 5k→100k, OpenAI − Titan	0.025	[−0.025, 0.075]	no	yes
Metadata lift at 5k, OpenAI − Titan	−0.005	[−0.045, 0.035]	yes	yes
Metadata lift at 50k, OpenAI − Titan	−0.025	[−0.070, 0.015]	no	yes
Metadata lift at 100k, OpenAI − Titan	0.005	[−0.030, 0.045]	yes	yes
Raw Hit@10 level at 100k, OpenAI − Titan	−0.025	[−0.075, 0.020]	no	yes
Raw drop, OpenAI − BM25	0.085	[0.025, 0.145]	no	no
Raw drop, Titan − BM25	0.050	[−0.005, 0.105]	no	no
Lift change 100k−5k, OpenAI n=200	0.000	[−0.040, 0.040]	yes	yes
Lift change 100k−5k, Titan n=200	−0.010	[−0.055, 0.035]	no	yes
Lift change 100k−5k, OpenAI n=400	−0.0175	[−0.045, 0.010]	yes	yes
Lift slope per tenfold N, OpenAI n=400	−0.017	[−0.041, 0.006]	yes	yes

earlier `ef_search` ablation. Table 16 lists, for the dedicated per-size indexes, table and index size, build time, and median and 95th-percentile query latency. Table 17 gives the number of documents per source at every ladder size, including the sizes at which the small sources are exhausted. The complete per-cell plans, recalls, and latencies are in the artifacts named in Sec. V-F.

APPENDIX F CORRECTIONS TO THE EARLIER RELEASE

An earlier public release of this study (v1) is archived with the repository. This paper replaces it, and the following items of v1 were wrong or weaker than reported. (1) *Search mode.* v1 reported its curves from one HNSW index queried with ef_search=200; the planner chose an exact scan or the index depending on database state (Sec. V-F), so v1 mixed exact and approximate search. Its OpenAI lift of +0.130 at 100k is +0.080 under exact search, its early step 0.795→0.660 is 0.795→0.735, and its Titan arm was in effect exact. (2) *The* ef_search *ablation* (+0.075 from 200 to 400) was a switch to an exact scan. (3) *The* n=400 *bank* used cached query vectors that belonged to other questions for 40 metadata questions: 0.710→0.468 becomes 0.7725→0.5675 (Sec. V-J). (4) *BM25* was truncated at 4k characters, text the dense model never saw; it is now text-matched (Sec. V-H). (5) *The oracle filter* kept only the first listed source type; it now uses all listed types (Appendix A). (6) *The threshold* $N^\star$ proved unstable and is not reported as a finding, and the description of the lift as embedder-dependent was an artifact of the approximate baseline (Appendix A).

ACKNOWLEDGMENT

The author used artificial intelligence (AI) coding and writing assistants—Cursor Composer, xAI Grok, Anthropic Claude, and Google Gemini—for literature search assistance, LaTeX drafting support, and manuscript reorganization (Introduction through Conclusion structure and wording drafts). In the revision of this paper, AI coding agents (Cursor Composer, xAI Grok 4.7, and Anthropic Claude) implemented the analysis, simulation, table-generation, and validation scripts from written specifications; second models (Google Gemini 4, OpenAI

TABLE 14. Share of the 200 primary questions whose plan used the HNSW index, for each embedder, condition (raw; filtered by the first listed source type, the v1 filter whose HNSW runs are audited), database (orig.: frozen original copy; fresh: fresh restore), and size. Each cell lists the share at `ef_search`=200/400/800 (1: every query used HNSW; 0: every query ran an exact scan). Every cell plans all 200 questions (plain `EXPLAIN`); the `EXPLAIN (ANALYZE, BUFFERS)` audit gives identical shares at `ef_search`=200 and agrees on its 40-question sample at 400 and 800. All other queries used a B-tree or sequential scan followed by an exact sort.

	OpenAI raw		OpenAI filtered		Titan raw		Titan filtered	
Scale	orig.	fresh	orig.	fresh	orig.	fresh	orig.	fresh
5k	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
10k	1/0/0	0/0/1	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
15k	1/0/0	0/0/1	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
20k	1/0/0	0/0/1	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
25k	1/0/0	0/0/1	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
40k	1/0/0	0/0/1	.44/0/0	0/0/.13	0/0/0	0/0/0	0/0/0	0/0/0
50k	1/0/0	0/0/1	.51/0/0	0/0/.44	0/0/0	0/0/0	0/0/0	0/0/0
75k	1/0/0	1/0/1	.80/0/0	0/0/.51	1/0/1	0/0/1	0/0/.60	0/0/.51
100k	1/0/0	1/0/1	.80/0/0	0/0/.60	1/0/1	1/0/1	0/0/.60	0/0/.60

TABLE 15. OpenAI raw @100k on the original database copy: planner behavior (Recall is recall@10 against exact search; Approx. and Exact are Hit@10 with approximate and exact search). Forced is Hit@10 with the HNSW index forced, from `hnsw_recall_by_cell_5434.json`, measured at `ef` $\in \{200, 400, 800\}$; a dash means not measured. At `ef` $\in \{100, 200, 300\}$ the planner used the index, so Approx. is an HNSW result; at `ef` $\geq$ 400 it chose an exact scan.

ef	Plan	Recall	Approx.	Exact	Forced
100	HNSW	0.893	0.495	0.585	–
200	HNSW	0.946	0.510	0.585	0.510
300	HNSW	0.967	0.530	0.585	–
400	exact	1.000	0.585	0.585	0.545
500	exact	1.000	0.585	0.585	–
600	exact	1.000	0.585	0.585	–
800	exact	1.000	0.585	0.585	0.565

GPT-5.6, and xAI Grok 4.7) reviewed scripts and results (they found and led to the correction of several defects, among them forced-index experiments that silently ran exact scans); and an AI assistant (Anthropic Claude Sonnet 5.5 and Claude Opus 5.5, used through Claude Code) coordinated the work and drafted the revised text of all sections and appendices. The author conceived and directed the study, chose the research questions, decided which controls to run, and interpreted the results, discussing directions and ideas with an AI assistant throughout; the validation checks were written by AI agents and reviewed by the author. Every number in the tables of this paper is generated from the published artifact files by script. Language models (OpenAI `gpt-4o` and GPT-5.6 Terra, Google Gemini 4, xAI Grok 4.7) were used as judges in the L3/L4 audits and the displaced-document audit (Sec. VII); these audits are model-judged corroboration, not human labeling. AI systems are not authors, and the author takes full responsibility for the content of this paper. All scientific claims, experimental design choices, reported primary (L1) metrics, exact-control numbers, and final wording were reviewed by the author and independently checked by AI reviewers and automated validators; the research idea and study protocol are the author's own. This disclosure follows IEEE guidance on AI-assisted manuscripts [35].

REFERENCES

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6769–6781, doi: 10.18653/v1/2020.emnlp-main.550.

[3] J. Ni *et al.*, "Large dual encoders are generalizable retrievers," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2022, pp. 9844–9855, doi: 10.18653/v1/2022.emnlp-main.669.

[4] Y. Fang, J. Zhan, Q. Ai, J. Mao, W. Su, J. Chen, and Y. Liu, "Scaling laws for dense retrieval," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2024, pp. 1339–1349, doi: 10.1145/3626772.3657743.

[5] J. Ning, Y. Kong, Y. Long, and J. Callan, "Less LLM, more documents: Searching for improved RAG," 2025, *arXiv:2510.02657*.

[6] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, "Filtered approximate nearest neighbor search in vector databases: System design and performance analysis," 2026, *arXiv:2602.11443*.

[7] D. Lu *et al.*, "An in-depth study of filter-agnostic vector search on a PostgreSQL database system: Experiments and analysis," 2026, *arXiv:2603.23710*.

[8] S. M. Shahri, M. Prammer, J. M. Patel, and A. Sriraman, "Analyzing and reducing search quality differences in vector similarity search," 2026, *arXiv:2608.25185*.

[9] L. Medrano, A. Verma, and M. Chhabra, "Scaling retrieval augmented generation with RAG fusion: Lessons from an industry deployment," 2026, *arXiv:2603.02153*.

[10] Y. Sun, J. Rahmfeld, C. Weaver, W. Chen, R. Desai, W. Huang, and M. H. Butler, "EnterpriseRAG-Bench: A RAG benchmark for company internal knowledge," 2026, *arXiv:2605.05253*.

[11] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks (NeurIPS)*, 2021.

[12] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in *Proc. 17th Conf. Eur. Chapter Assoc. Comput. Linguistics (EACL)*, 2023, pp. 2014–2037, doi: 10.18653/v1/2023.eacl-main.148.

[13] N. Reimers and I. Gurevych, "The curse of dense low-dimensional information retrieval for large index sizes," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics (ACL), Short Papers*, 2021, pp. 605–611, doi: 10.18653/v1/2021.acl-short.77.

[14] O. Weller, M. Boratko, I. Naim, and J. Lee, "On the theoretical limitations of embedding-based retrieval," 2025, *arXiv:2508.21038*.

[15] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,"

TABLE 16. Per-*N* index capacity and query latency (raw), with build times from per-scale HNSW builds.

Embedder	*N*	Rows	Total MB	HNSW MB	Build s	p50@200 (ms)	p95@200 (ms)	p50@800 (ms)	p95@800 (ms)
OpenAI	5k	5000	83.411	40.968	2.160	3.221	3.985	5.787†	6.724†
OpenAI	10k	10000	166.707	81.928	4.868	4.035	5.500	7.816†	9.485†
OpenAI	15k	15000	249.905	122.888	7.742	3.272	4.274	8.234	10.376
OpenAI	20k	20000	333.226	163.848	11.074	3.820	5.122	7.879	10.128
OpenAI	25k	25000	416.612	204.808	14.256	3.282	4.289	7.445	9.559
OpenAI	40k	39999	666.345	327.680	25.039	3.801	5.100	8.610	11.685
OpenAI	50k	49999	833.020	409.600	30.672	3.931	5.204	9.797	12.967
OpenAI	75k	74999	1249.190	614.384	48.933	4.078	5.535	9.749	12.861
OpenAI	100k	99998	1666.089	819.175	16.859	4.308	6.220	10.857	14.639
Titan	5k	5000	69.591	40.968	1.573	2.773	3.305	6.147†	7.048†
Titan	10k	10000	139.100	81.928	3.599	3.144	3.958	7.374†	8.452†
Titan	15k	15000	208.617	122.888	6.331	3.469	4.482	7.302	8.859
Titan	20k	20000	278.127	163.848	8.737	3.503	4.516	8.302	10.419
Titan	25k	25000	347.726	204.808	11.542	3.450	4.423	8.496	10.650
Titan	40k	39999	556.138	327.680	19.205	3.426	4.667	8.382	10.880
Titan	50k	49999	695.296	409.600	27.336	3.013	4.030	9.318	12.257
Titan	75k	74999	1042.760	614.384	39.501	3.797	4.955	10.174	13.170
Titan	100k	99998	1390.420	819.175	13.239	4.063	5.374	10.438	13.780

Host shared with other processes; query latency indicative only. Latencies are the median and 95th percentile over questions of per-question medians from three timed 200-question sweeps (`relaxed_order`, `max_scan_tuples`=20000). †Default plan used sequential scan; latency from matching `seqscan_off` twin (the same probes run with `SET enable_seqscan=off`; 4 cells). "Total MB" is on-disk size, `total_bytes`/10^6 (heap, TOAST, and indexes), not memory use; the measured memory working set is given in Sec. V-G. The HNSW size is the same for both embedders because each index entry occupies one 8 kB page at these dimensions. Exact scans are not timed here; the exact scan of the 200 queries at 100k took 43.7 s on the original database (artifact `exact_control_mvp`). The 5k heap is small because vectors are stored out of line. Build time is the primary build (session setting `max_parallel_maintenance_workers`= 4, server default 2); four further OpenAI builds per size took 2.2–2.3 s at 5k, 50.2–56.0 s at 75k and 25.1–29.9 s at 100k. The 100k builds are faster because PostgreSQL assigns parallel build workers only when the heap reaches 8 MB (`min_parallel_table_scan_size`), which happens between 75k (881 pages) and 100k (1,175); builds up to 75k ran serially (a 75k copy built in 50.6 s serially and in 12.2 s with workers allowed; artifact `build_parallel_probe`).

TABLE 17. Documents per source at every ladder size under the round-robin schedule (gold anchors included; sizes count documents with `scale_rank`<*N*, so 49,999 and 99,998 occur because two ranks are unused). A source stops growing once it is exhausted.

Source	5k	10k	15k	20k	25k	40k	50k	75k	100k
confluence	683	1,239	1,794	2,350	2,907	4,572	5,189	5,189	5,189
fireflies	518	1,074	1,629	2,185	2,740	4,407	5,580	9,263	10,173
github	551	1,107	1,662	2,218	2,773	4,440	5,613	8,052	8,052
gmail	504	1,060	1,615	2,171	2,726	4,393	5,566	9,249	14,067
google_drive	559	1,114	1,670	2,226	2,781	4,448	5,621	9,304	14,122
hubspot	536	1,091	1,647	2,202	2,758	4,425	5,598	9,281	14,098
jira	596	1,151	1,707	2,262	2,817	4,484	5,656	6,118	6,118
linear	554	1,109	1,665	2,220	2,776	4,442	5,615	9,299	14,117
slack	499	1,055	1,611	2,166	2,722	4,388	5,561	9,244	14,062
Total	5,000	10,000	15,000	20,000	25,000	39,999	49,999	74,999	99,998

IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, Apr. 2020, doi: 10.1109/TPAMI.2018.2889473.

[16] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021, doi: 10.1109/TBDATA.2019.2921572.

[17] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019.

[18] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Inf. Syst.*, vol. 87, Art. no. 101374, Jan. 2020, doi: 10.1016/j.is.2019.02.006.

[19] J. Wang *et al.*, "Milvus: A purpose-built vector data management system," in *Proc. Int. Conf. Manage. Data (SIGMOD)*, 2021, pp. 2614–2627, doi: 10.1145/3448016.3457550.

[20] S. Gollapudi *et al.*, "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conf. (WWW)*, 2023, pp. 3406–3416, doi: 10.1145/3543507.3583552.

[21] J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," *VLDB J.*, vol. 33, no. 5, pp. 1591–1615, 2024, doi: 10.1007/s00778-024-00864-x.

[22] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," *Proc. ACM Manage. Data*, vol. 2, no. 3, pp. 1–27, May 2024, doi: 10.1145/3654923.

[23] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics: System Demonstrations*, 2024, pp. 150–158, doi: 10.18653/v1/2024.eacl-demo.16.

[24] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," 2023, *arXiv:2312.10997*.

[25] H. Zeng, J. Killingback, and H. Zamani, "Scaling sparse and dense retrieval in decoder-only LLMs," 2025, *arXiv:2502.15526*.

[26] A. Kane, "pgvector: Open-source vector similarity search for Postgres,"

GitHub repository. Accessed: Nov. 11, 2026. [Online]. Available: https://github.com/pgvector/pgvector
[27] OpenAI, "New embedding models and API updates," Jan. 2024. Accessed: Nov. 11, 2026. [Online]. Available: https://openai.com/index/new-embedding-models-and-api-updates/
[28] Amazon Web Services, "Amazon Titan Text Embeddings models," *Amazon Bedrock User Guide*. Accessed: Nov. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html
[29] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall/CRC, 1994, doi: 10.1201/9780429246593.
[30] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 4, nos. 1–2, pp. 1–174, 2009, doi: 10.1561/1500000019.
[31] M. D. Smucker, J. Allan, and B. Carterette, "A comparison of statistical significance tests for information retrieval evaluation," in *Proc. 16th ACM Conf. Inf. Knowl. Manage. (CIKM)*, 2007, pp. 623–632, doi: 10.1145/1321440.1321528.
[32] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2009, pp. 758–759, doi: 10.1145/1571941.1572114.
[33] T. Chen *et al.*, "Dense X retrieval: What retrieval granularity should we use?" in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2024, pp. 15159–15177, doi: 10.18653/v1/2024.emnlp-main.845.
[34] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2020, pp. 39–48, doi: 10.1145/3397271.3401075.
[35] IEEE, "Author guidelines for artificial intelligence (AI)-generated text," Apr. 2024. Accessed: Nov. 11, 2026. [Online]. Available: https://open.ieee.org/author-guidelines-for-artificial-intelligence-ai-generated-text/



BHARGAVA CHARY PEDDAPUDI received the M.S. degree in applied computer science from Northwest Missouri State University, Maryville, MO, USA, in 2015. He is an independent researcher based in the San Francisco Bay Area, CA, USA. His research interests include retrieval-augmented generation (RAG) evaluation, metadata-aware vector retrieval, and scaling behavior of approximate nearest-neighbor indexes.